

COMPETITIVE PROGRAMMING TEMPLATE

STAY HUNGRY, STAY FOOLISH

by

TLE_Automat

NEUQ_ACM

字符串部分

一. 字符串通用算法

I. 字符串哈希

二. 字符串匹配相关

I. kmp 算法

II. exkmp 算法 (Z 算法)

III. kmp 自动机

三. 回文相关

I. manacher 算法 (马拉车算法)

II. 回文自动机 (PAM)

四. 字符串后缀数据结构

I. AC 自动机

II. 后缀数组 (Suffix Array)

III. 后缀自动机 (Suffix AutoMaton)

IV. 广义后缀自动机

数据结构与分治部分

一. 树形数据结构

I. 树状数组

II. 可持久化线段树

III. 线段树合并

IV. 李超线段树

V. 动态开点线段树求解线段覆盖问题

VI. Treap

VII. 笛卡尔树的线性构建

VIII. `__gnu_pbds::tree` —— C++ 扩展库中的平衡树使用方法

IX. 并查集

二. 分治相关

I. cdq 分治

II. 整体二分

三. 其他数据结构

I. ST 表 (求解区间 RMQ)

图论部分

一. 最短路与最小生成树进阶

I. 同余最短路

II. kruskal 重构树

二. 连通性相关

I. tarjan 算法求强连通分量并缩点

II. tarjan 算法求边双连通分量并缩点

III. tarjan 算法求点双连通分量并建立圆方树

IV. 利用强连通分量求解 2-sat 问题

三. 图上计数相关

I. 无向图三元环计数

四. 网络流相关

I. Dinic 算法求最大流

II. SSP 算法求最小费用最大流

五. 树上问题

I. 树上启发式合并 (Dsu on tree)

II. 轻重链剖分

III. 树哈希

IV. 点分治

六. 图论杂项

I. 欧拉路的判定与求解

II. 平面图的判定

III. 最小斯坦纳树

数学部分

一. 数论基础

I. 扩展欧几里得算法 (exgcd)

II. 扩展中国剩余定理 (exCRT)

III. 欧拉定理与扩展欧拉定理

IV. 阶与原根

定义

性质

证明

求原根算法流程:

V. 离散对数

二. 数论进阶——素数、数论函数与筛法

I. 线性筛

II. 狄利克雷卷积与莫比乌斯反演的一些结论

反演过程

常用数论函数

常用结论

III. 杜教筛

解决问题

解决方法

时间复杂度

三. 线性代数

I. 线性基

四. 数值算法

I. 拉格朗日插值

前置知识

结论推导

一个例题

五. 组合数学

I. 组合数的基本性质

排列数与组合数

杨辉三角、组合数的一些恒等式、二项式定理

范德蒙德卷积

卡特兰数

II. 容斥原理

III. 二项式反演

III. 生成函数初步

概率生成函数 PGF

IV. 十二重计数法

第六重——第二类斯特林数

1. 递推式

2. 通项式

3. 行的计算

4. 列的计算

5. 应用

六. 计算几何

I. 向量基础运算

II. 多边形

III. 二维凸包

七. 多项式部分

I. 多项式求逆

II. 多项式 $\ln$

III. 多项式 $\exp$

IV. 多项式封装化模板

V. 任意模数 NTT

MISC (杂项)

mt19937-随机数

扫描线

Fwt 快速沃尔什变换

字符串部分

一. 字符串通用算法

I. 字符串哈希

字符串哈希封装模板（双模数哈希版本）

```
1 struct Hash {
2     const int Mod[2] = {(int)1e9 + 7, 998244353};
3     const int base[2] = {233, 177};
4
5     vector<int> hsh[2], pw[2];
6     void init(string s, int n) { // index from [1]
7         for (int x = 0; x < 2; x++) {
8             hsh[x].resize(n + 1);
9             pw[x].resize(n + 1);
10        }
11        pw[0][0] = pw[1][0] = 1;
12        for (int i = 1; i <= n; i++) {
13            for (int x = 0; x < 2; x++) {
14                pw[x][i] = 111 * pw[x][i - 1] * base[x] % Mod[x];
15                hsh[x][i] = (111 * hsh[x][i - 1] * base[x] + s[i]) %
Mod[x];
16            }
17        }
18    }
19    pii get_hsh(int l, int r) {
20        return {(hsh[0][r] - 111 * hsh[0][l - 1] * pw[0][r - l + 1] %
Mod[0] + Mod[0]) % Mod[0],
21                (hsh[1][r] - 111 * hsh[1][l - 1] * pw[1][r - l + 1] %
Mod[1] + Mod[1]) % Mod[1]};
22    }
23    bool qsame(int l1, int r1, int l2, int r2) {
24        return get_hsh(l1, r1) == get_hsh(l2, r2);
25    }
26 } preH, revH;
```

二. 字符串匹配相关

I. kmp 算法

考虑定义 border 集合，设字符串 T 满足：

- $T \neq S$ 。
- T 是 S 的前缀， T 是 S 的后缀。

则所有满足条件的字符串 T 称为字符串 S 的 border 集合。

对于字符串 S ，定义其 nxt 数组为， nxt_i 表示 S 的长度为 i 的前缀的 border 集合中最长字符串的长度。

显然暴力构造 nxt 数组时间复杂度为 $O(n^2)$ 。

考虑 nxt 数组的性质：

- $nxt_1 = 0$ 。
- $nxt_{i+1} \leq nxt_i + 1$ 。

考虑第二点性质是为什么，因为 nxt_{i+1} 所表示的最长长度的 $S[1..i+1]$ 的 border 中的字符串，在最长情况下只能是：在 nxt_i 所表示的 $S[1..i]$ 的 border 中的最长字符串所在的前缀的后面一个字符，恰好和 S_{i+1} 相同。这才能使得 $nxt_{i+1} = nxt_i + 1$ 。

如果 nxt_{i+1} 能更大的话，考虑去掉最后一个字符得到 nxt_i ，这样就会发现我们现在得到的 nxt_i 比之前求出的大，就矛盾了。

根据这条性质，我们可以暴力跳 nxt 数组，以求出当前位置的 nxt ，正确性证明同上，代码如下：

```
1 void get_nxt(char *s, int n) {
2     fail[1] = 0;
3     for (int i = 2; i <= n; i++) {
4         int pos = fail[i - 1];
5         while (pos && s[pos + 1] != s[i]) {
6             pos = fail[pos];
7         }
8         if (s[pos + 1] == s[i]) {
9             pos++;
10        }
11        fail[i] = pos;
12    }
13 }
```

其中，border 还有一个非常厉害的结论：

对于字符串 S ，如果满足：

$$S \equiv 0 \pmod{|S| - |\text{max_border}(S)|}$$

则 $|S| - |\text{max_border}(S)|$ 就是 S 的**最小循环节长度**，否则 S 的最小循环节长度就是 $|S|$ 。

证明的话，就是考虑一段一段对应平移，能证出是循环节，最小的话考虑反证。

II. exkmp 算法 (Z 算法)

exkmp 封装模板

```
1 struct Exkmp {
2     int n, m;
3     string s, t;    // string (Index from 1)
4     vector<int> z, zt;
5     /*
6     - z   : s 与自己的每个后缀的 lcp 长度
7     - zt  : s 与 t 的每个后缀的 lcp 长度
8     */
9     Exkmp(string s): n(SZ(s)), s(" " + s) {
10         z.resize(n + 1, 0);
11         zwork();
12     }
13     Exkmp(string s, string t): n(SZ(s)), m(SZ(t)), s(" " + s), t(" " + t) {
14         z.resize(n + 1, 0);
15         zt.resize(m + 1, 0);
16         zwork();
17         ztwork();
18     }
19     void zwork() {
20         z[1] = n;
21         int l = 0, r = 0;
22         for (int i = 2; i <= n; i++) {
23             if (i <= r) {
24                 z[i] = min(z[i - l + 1], r - i + 1);
25             }
26             while (i + z[i] <= n && s[z[i] + 1] == s[i + z[i]]) {
27                 z[i]++;
28             }
29             if (i + z[i] - 1 > r) {
30                 l = i;
31                 r = i + z[i] - 1;
32             }
33         }
34     }
35     void ztwork() {
36         int l = 0, r = 0;
37         for (int i = 1; i <= m; i++) {
38             if (i <= r) {
39                 zt[i] = min(z[i - l + 1], r - i + 1);
40             }
41             while (i + zt[i] <= m && zt[i] + 1 <= n && s[zt[i] + 1] == t[i
+ zt[i]]) {
42                 zt[i]++;
```

```
43         }
44         if (i + zt[i] - 1 > r) {
45             l = i;
46             r = i + zt[i] - 1;
47         }
48     }
49 }
50 };
```

III. kmp 自动机

可以被 AC 自动机/后缀自动机完全代替，故懒得学了。

三. 回文相关

I. manacher 算法 (马拉车算法)

manacher 算法 (马拉车算法) 封装模板

```
1 namespace Manacher {
2     string t;
3     vector<int> p;
4     void init(string s, int n) { // index from [1]
5         t.resize(2 * n + 1);
6         p.resize(2 * n + 1);
7
8         t[0] = '#';
9         for (int i = 1; i <= n; i++) {
10             t[2 * i - 1] = s[i];
11             t[2 * i] = '#';
12         }
13
14         int id = 0, mx = 0; p[0] = 1;
15         for (int i = 1; i <= 2 * n; i++) {
16             p[i] = (i < mx ? min(p[2 * id - i], mx - i) : 1);
17             while (i + p[i] <= 2 * n && i - p[i] >= 0 && t[i + p[i]] == t[i
- p[i]]) {
18                 p[i]++;
19             }
20             if (i + p[i] > mx) {
21                 id = i;
22                 mx = i + p[i];
23             }
24         }
25     }
26 }
```

II. 回文自动机 (PAM)

其实也是字符串后缀数据结构的一种，但还是先把放到回文部分里面了。

建议先理解透彻后缀自动机，再来学回文自动机，这样一些概念和性质会理解得非常快，事半功倍。

定义

回文自动机（回文树），一种有限状态自动机，一种可以存储一个串中所有回文子串的高效数据结构。可以简单高效地解决众多与字符串回文相关的问题。

记号与约定

下文中， s 是一个长为 n 字符串， s' 为 s 的倒串（翻转后的串）， $s[l, r]$ 是 s 的一个子串（下标从 1 开始）。

特殊情况见上下文说明。

一些性质

引理 1.1: 一个回文串 s 的 border 必定是一个回文串。

引理 1.2: 一个回文串 s 的所有回文后缀必定是它的 border。

证明 1.1:

- 钦定一个 s 的 border 为 $s[n - i + 1, n]$ 。
- 根据 border 的性质, $s[1, i] = s[n - i + 1, n]$; 根据 s 作为回文串的性质, $s[1, i] = s'[n - i + 1]$ 。故而有 $s[n - i + 1, n] = s'[n - i + 1, n]$, 而它恰好是回文串的定义。

证明 1.2: 可以把引理 1.1 的证明方法倒回去。

引理 2: 对于一个字符串 s , 它本质不同的回文子串一定是以某个点结尾的、最长的那个回文串。

证明 2: 考虑反证。假设本质不同的回文子串是以某个点结尾的, 但不是最长的回文串, 那么根据引理 1.2 以及 border 的性质 (s 的 border 的 border 也是 s 的 border), 它必定已被统计过了。

引理 3: s 的本质不同的子串个数不超过 n 个。

证明 3: 根据引理 2 可得。

结构

PAM 和 ACAM 类似，它可以用树的形式展现出来：Trie 树或 fail 树。

不难注意到按照传统的 Trie 树，已经难以去方便地处理回文串了。

故而考虑 Trie 树的变形：

- 初始时有两个节点，它们所代表的串一个长度为 0（用于处理偶回文），一个长度为 -1 （用于处理奇回文）。
- 对于一个节点连出的边权 c 的边，就等价于在当前节点所代表的串两边各加上一个字符 c 。

fail 树自然也跟着有了变化：

- 对于一个节点的 fail 指针，它指向的是当前节点的最长回文后缀。
- 其中为了后续处理方便，我们让串长为 0 的节点 fail 指向串长为 -1 的节点。

构建方式

我们采用地构建方法是逐个插入。逐个插入一般有两种：首端/末端插入。

考虑末端插入字符 c ：

1. 以 $i - 1$ 结尾的最长回文子串为开始，一直跳 fail 指针，直到跳到一个合法的节点或者不能再跳了。

“合法”的定义：一个节点是合法的当且仅当它所代表的串是合法的，一个串是合法的当且仅当位于它左边的字符是 c ，这样就可以形成一个以新插入字符结尾的回文串。

2. 判断当前跳到的节点是否是合法的节点。若是合法的节点，那就插入一个新节点，同时维护一下它的信息。不是就不插入。

- Note：会发现，对于为什么要像介绍 PAM 结构时提到的那样：“让串长为 0 的节点 fail 指向串长为 -1 的节点”，现在已经有了答案。

假设跳 fail 指针时，一直跳都没有找到合法的节点，且最后落到了串长为 0 的那个节点上。若那个串长为 0 的节点也是不合法的，这种情况显然是没法插入一个新节点，而实际上它应该插入一个长为 1 的串。

再考虑首端插入。根据引理 1.2，字符串的最长回文后缀等价于字符串的最长回文前缀，故而首端插入和末端插入如出一辙。

```

1 struct PAM {
2     string s;
3     int lst, tot, n;
4     vector<int> fa, len, sz, cnt, pos;
5     vector<array<int, 26>> ch;
6     vector<array<int, 21>> st;
7     /*
8     - len[i] : the length of the longest palindrome string end with i
9     - sz[i]  : the number of the palindrome string end with i
10    - cnt[i] : the occurrence number of the longest palindrome string end
with i
11    - fa[i]  : i 的回文后缀链接
12    - pos[i] : 下标为 i 的字符在回文树上对应的结点
13    - st     : fa 的倍增数组
14    */
15    PAM() {}
16    PAM(string s): s(s), n(s.length() + 2) { // String s (Index from 0)
17        fa.resize(n); len = sz = cnt = pos = fa;
18        ch = vector(n, array<int, 26>());
19        st = vector(n, array<int, 21>());
20        len[0] = 0; len[1] = -1;
21        fa[0] = 1; fa[1] = 0;
22        lst = 0; tot = 1;
23    }
24    int get_fail(int x, int now) {
25        while (s[now - len[x] - 1] != s[now]) {
26            x = fa[x];
27        }
28        return x;
29    }
30    void insert(int c, int now) {
31        int p = get_fail(lst, now);
32        if (!ch[p][c]) {
33            len[++tot] = len[p] + 2;
34            int tmp = get_fail(fa[p], now);
35            fa[tot] = ch[tmp][c];
36            sz[tot] = sz[fa[tot]] + 1;
37            ch[p][c] = tot;
38        }
39        lst = ch[p][c];
40        cnt[lst] += 1;
41        pos[now] = ch[p][c];
42    }
43    void init() {
44        for (int i = 0; i < SZ(s); i++) {
45            insert(s[i] - 'a', i);
46        }

```

```
47     for (int i = 0; i <= tot; i++) {
48         st[i][0] = fa[i];
49     }
50     for (int j = 1; j <= 20; j++) {
51         for(int i = 0; i <= tot; i++) {
52             st[i][j] = st[st[i][j - 1]][j - 1];
53         }
54     }
55 }
56 };
```

四. 字符串后缀数据结构

I. AC 自动机

AC 自动机模板封装

```
1 struct AC_Automat {
2     int cnt_id;
3     int tr[MAXN][26];
4     int fail[MAXN];
5
6     void insert(string s, int n) { // index from 1
7         int u = 0;
8         for (int i = 1; i <= n; i++) {
9             if (!tr[u][s[i] - 'a']) {
10                 tr[u][s[i] - 'a'] = ++cnt_id;
11             }
12             u = tr[u][s[i] - 'a'];
13         }
14     }
15
16     void build() {
17         queue<int> q;
18         for (int i = 0; i < 26; i++) {
19             if (tr[0][i]) {
20                 q.push(tr[0][i]);
21             }
22         }
23         while (!q.empty()) {
24             int u = q.front();
25             q.pop();
26             for (int i = 0; i < 26; i++) {
27                 if (tr[u][i]) {
28                     fail[tr[u][i]] = tr[fail[u]][i];
29                     q.push(tr[u][i]);
30                 }
31                 else {
32                     tr[u][i] = tr[fail[u]][i];
33                 }
34             }
35         }
36     }
37 } ac;
```

II. 后缀数组 (Suffix Array)

后缀数组封装模板

```
1 namespace SA {
2     int n, m;
3     char s[MAXN];
4     int sa[MAXN], tmp[MAXN], tong[MAXN];
5     int rk[MAXN << 1], lst_rk[MAXN << 1];
6     int height[MAXN];
7
8     void rsort() {
9         memset(tong, 0, sizeof(tong));
10        for (int i = 1; i <= n; i++) tong[rk[tmp[i]]]++;
11        for (int i = 1; i <= m; i++) tong[i] += tong[i - 1];
12        for (int i = n; i >= 1; i--) sa[tong[rk[tmp[i]]]--] = tmp[i];
13    }
14
15    void get_sa() {
16        m = 133;
17        for (int i = 1; i <= n; i++) rk[i] = s[i];
18        for (int i = 1; i <= n; i++) tmp[i] = i;
19        rsort();
20
21        for (int w = 1; w < n; w <= 1) {
22            int p = 0;
23            for (int i = n - w + 1; i <= n; i++) tmp[++p] = i;
24            for (int i = 1; i <= n; i++) if (sa[i] > w) tmp[++p] = sa[i] -
w;
25            rsort();
26
27            p = 0;
28            memcpy(lst_rk, rk, sizeof(rk));
29            for (int i = 1; i <= n; i++) {
30                if (lst_rk[sa[i] - 1] == lst_rk[sa[i]] && lst_rk[sa[i] - 1]
+ w == lst_rk[sa[i] + w]) rk[sa[i]] = p;
31                else rk[sa[i]] = ++p;
32            }
33            m = p;
34
35            if (m == n) break;
36        }
37    }
38
39    void get_height() {
40        int len = 0;
41        for (int i = 1; i <= n; i++) {
```

```

42         if (rk[i] == 1) continue;
43         if (len) len--;
44         int j = sa[rk[i] - 1];
45         while (i + len <= n && j + len <= n && s[i + len] == s[j +
len]) len++;
46         height[rk[i]] = len;
47     }
48 }
49
50 void init(char *t, int len) {
51     n = len;
52     for (int i = 1; i <= n; i++) {
53         s[i] = t[i];
54     }
55     get_sa();
56     get_height();
57 }
58 } using SA::sa;

```

III. 后缀自动机 (Suffix AutoMaton)

关于后缀自动机的一些总结。

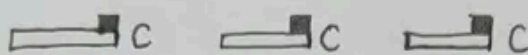
1. 字符串 s 的 SAM 是接受 s 的所有后缀的最小 DFA。
2. 记 $\text{endpos}(t)$ 为 t 在字符串 s 中所有结束位置的集合，则可以将 s 的所有子串根据 endpos 划分成若干个等价类。两个不同的 endpos 集合要么交集为空集，要么一个被另一个包含。
3. 令空串的 endpos 为 $\{1, 2, \dots, n\}$ ，则每次向这个串的串首添加 Σ 中不同的字符并保存，则相当于把 endpos 做了分裂，这样一直添加下去会形成一个树形结构（这就是后缀树，可以用线段树合并求出每个 endpos 集合中的具体元素），每个点相当于一个 endpos 集合，兄弟结点的集合之间没有交集，父亲结点的集合相当于所有儿子结点的集合的并再并上分裂时丢失的位置（分裂时可能会丢失一些位置是因为，这些位置的前缀已经顶到 s 的头了，没法再串首添加新的字符了）。
4. 由于分裂到最后，每个叶子结点代表的 endpos 集合中至少有一个位置，且互不相交，所以叶子结点最多有 n 个。如果一个父亲只有一个儿子，那这个结点的集合在分裂时一定丢失了一些位置，否则一个父亲至少有两个儿子，所以树的总结点个数小于等于 $2n - 1$ ，即只有 $O(n)$ 种不同的 endpos 集合（等价类）。
5. SAM 的每个结点实际就是一个 endpos 集合，SAM 中的转移边描述的就是如何在一个 endpos 集合后添加 Σ 中的一个字符转移到另外一个 endpos 集合。
6. 对于每个结点 u 代表的 endpos 集合记录一个 len_u 代表这个集合分出的等价类中最长的串的长度，显然最短的长度就是 $\text{len}_{f_{a_u}} + 1$ （注意长度一定是连续的，证明比较显然），如果匹配过程中走到某个 $\text{len}_u = n$ 的结点的祖先结点上，就说明一个后缀匹配成功了。

转移边分为两种 (从状态 p 经过一个字符 c 的转移边到状态 q)

① $len_q = len_p + 1$ (称为“连续转移”)

② $len_q > len_p + 1$ (称为“不连续转移”)

连续转移举例



“ $\overbrace{adc} \overbrace{adc} \overbrace{bdc}$ ” (从 $ep("d")$ 到 $ep("dc")$ 的转移)

不连续转移举例



(,  的公共后缀长度为 0)

“ $\overbrace{adc} \overbrace{adc} \overbrace{bd}$ ” (从 $ep("d")$ 到 $ep("dc") = ep("adc")$, len 从 1 变为 3)

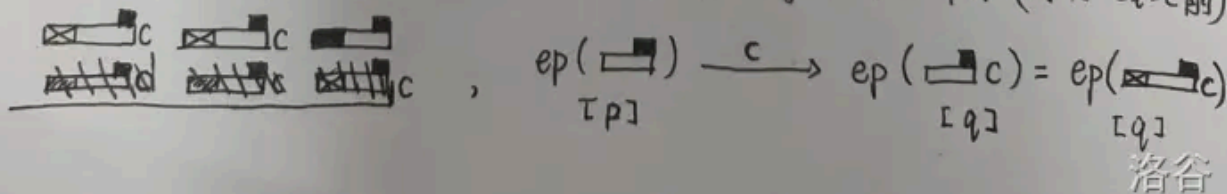
从上面的结构可以推出, 在后缀自动机构建的过程中,

沿着 $last$ 的后缀链接一直跳, 记跳到的状态为 p , 且 p 有字符 c 的转移, 令 $q = ntp_{p,c}$.

① 若未找到有字符 c 转移的 p , 令 $link_{cur} = 0$. $nxt_{last,c} = cur$ ~~$nxt_{last,c} = cur$~~ (for all)

② 若从 p 到 q 为连续转移, 相当于在 $ep(\overbrace{\hspace{1cm}}c)$ 中 ^向 添加了一个位置 ~~i~~ , 令 $link_{cur} = q$, 然后退出。

③ 若从 p 到 q 为不连续转移, 转移图如下所示 (添加 $s_c = c\overline{q}$)



注意到添加 $S_i = c$ 后, $ep(\boxed{}c) \neq ep(\boxed{\times}c)$ 。

原本的结构是

$$\underset{[p]}{ep(\boxed{})} \xrightarrow{c} ep(\boxed{}c) = ep(\boxed{\times}c) \underset{[q]}{}$$

现在的结构是

(波浪线代表“后缀链接”)

$$\underset{[p]}{ep(\boxed{})} \xrightarrow{c} ep(\boxed{}c) \underset{[clone]}{}$$

$$\quad \quad \quad \uparrow \text{fa}$$

$$\quad \quad \quad ep(\boxed{\times}c) \underset{[q]}{}$$

于是非常显然, 克隆 q ^结点的除 len 之外的所有信息到新结点 $clone$,

令 $len_{clone} = len_p + 1$, $link_{cur} = link_q = clone$,

然后 ~~遍历 q 的所有后缀链接~~

沿着后缀链接从 p 往回走, 若经过状态存在向 q 的转移, 将转移重新连接到 $clone$ 。

SAM 中转移数是线性的证明:

连续的转移一定形成一棵树 (首先所有前缀一定是一个连续的转移, 且

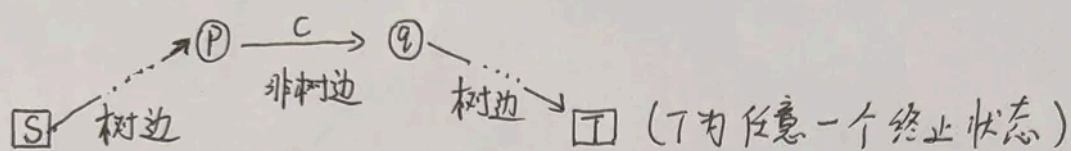
不存在这种结构 $\textcircled{p_1} \xrightarrow{\text{连续转移}} \textcircled{q}$, $\textcircled{p_2} \xrightarrow{\text{连续转移}} \textcircled{q}$, 因 $len_q = len_{p_1} + 1 = len_{p_2} + 1$, $len_{p_1} = len_{p_2}$

且 $endpos$ 有交, 那么 $\textcircled{p_1}$ 和 $\textcircled{p_2}$ 必定完全相同)

所以连续的转移不会超过 $2(n-1)$ 条,

洛谷

非连续的转移相当于连续转移形成的树上的一些非树边，
将这条非树边进行映射为如下路径代表的字符串。



映射成的字符串一定是 S 的一个后缀。又由于不同的非树边映射的路径一定不同，SAM 上不同的路径一定形成不同的字符串，而 S 最多只有 $n-1$ 个不等于自己的后缀，所以非连续转移最多只有 $n-1$ 条。

洛谷

```

1 namespace SAM {
2     int fa[MAXN << 1], mxlen[MAXN << 1];
3     int nxt[MAXN << 1][26];
4
5     int cnt_id, lst;
6     int epsz[MAXN << 1];
7
8     void ins(char ch) {
9         int c = ch - 'a';
10        int cur = cnt_id++;
11
12        epsz[cur] = 1;
13        mxlen[cur] = mxlen[lst] + 1;
14
15        int p = lst;
16        for (; p != -1 && !nxt[p][c]; p = fa[p]) {
17            nxt[p][c] = cur;
18        }
19
20        if(p == -1) {
21            fa[cur] = 0;
22        }
23        else {
24            int q = nxt[p][c];
25            if (mxlen[p] + 1 == mxlen[q]) {
26                fa[cur] = q;
27            } else {
28                int clone = cnt_id++;
29                fa[clone] = fa[q];
30                mxlen[clone] = mxlen[p] + 1;
31                memcpy(nxt[clone], nxt[q], sizeof(nxt[q]));
32
33                fa[cur] = fa[q] = clone;
34                for (; p != -1 && nxt[p][c] == q; p = fa[p]) {
35                    nxt[p][c] = clone;
36                }
37            }
38        }
39        lst = cur;
40    }
41
42    vector<int> G[MAXN << 1];
43    void dfs(int u) {
44        for (auto v : G[u]) {
45            dfs(v);

```

```

46         epsz[u] += epsz[v];
47     }
48 }
49
50 void build(string s, int n) {           // indexed from 1
51     cnt_id = 1; lst = 0;
52     fa[0] = -1; mxlen[0] = 0;
53     for (int i = 1; i <= n; i++) {
54         ins(s[i]);
55     }
56     for (int i = 1; i < cnt_id; i++) {
57         G[fa[i]].push_back(i);
58     }
59     dfs(0);
60 }
61 }

```

IV. 广义后缀自动机

```
1 struct exSAM {
2     int mxlen[(N << 1) + 1], fa[(N << 1) + 1];
3     int nxt[(N << 1) + 1][26];
4     int cnt_id;
5
6     void init() {
7         cnt_id = 1;
8         fa[0] = -1;
9     }
10
11     int insert(int lst, int c) {
12         int cur = nxt[lst][c];
13         if (mxlen[cur]) {
14             return cur;
15         }
16         mxlen[cur] = mxlen[lst] + 1;
17
18         int p = fa[lst];
19         for (; p != -1 && !nxt[p][c]; p = fa[p]) {
20             nxt[p][c] = cur;
21         }
22         if (p == -1) {
23             fa[cur] = 0;
24             return cur;
25         }
26
27         int q = nxt[p][c];
28         if (mxlen[p] + 1 == mxlen[q]) {
29             fa[cur] = q;
30             return cur;
31         }
32
33         int clone = cnt_id++;
34         fa[clone] = fa[q];
35         mxlen[clone] = mxlen[p] + 1;
36         for (int i = 0; i < 26; i++) {
37             nxt[clone][i] = (mxlen[nxt[q][i]] ? nxt[q][i] : 0);
38         }
39
40         fa[cur] = fa[q] = clone;
41         for (; p != -1 && nxt[p][c] == q; p = fa[p]) {
42             nxt[p][c] = clone;
43         }
44         return cur;
45     }
```

```

46
47     void add_str(string s, int n) {          // index from 1
48         int u = 0;
49         for (int i = 1; i <= n; i++) {
50             int c = s[i] - 'a';
51             if (!nxt[u][c]) {
52                 nxt[u][c] = cnt_id++;
53             }
54             u = nxt[u][c];
55         }
56     }
57
58     void build() {
59         queue<pii> q;
60         for (int i = 0; i < 26; i++) {
61             if (nxt[0][i]) {
62                 q.push({0, i});
63             }
64         }
65         while (!q.empty()) {
66             auto [lst, c] = q.front();
67             q.pop();
68             int cur = insert(lst, c);
69             for (int i = 0; i < 26; i++) {
70                 if (nxt[cur][i]) {
71                     q.push({cur, i});
72                 }
73             }
74         }
75     }
76 } sam;

```

数据结构与分治部分

一. 树形数据结构

I. 树状数组

树状数组封装模板

```
1  template<typename T>
2  struct BIT {
3      int lim, p;
4      vector<T> d;
5      void init(int n) {
6          d.clear();
7          lim = 1, p = 0;
8          while (lim < n) {
9              lim <<= 1;
10             p++;
11         }
12         d.resize(lim + 1, 0);
13     }
14     void upd(int x, int v) {
15         for (; x <= lim; x += lb(x)) {
16             d[x] += v;
17         }
18     }
19     T query(int x) {
20         T res = 0;
21         for (; x; x -= lb(x)) {
22             res += d[x];
23         }
24         return res;
25     }
26     T query(int l, int r) {
27         return query(r) - query(l - 1);
28     }
29     T kth(int k) { // 查找第 k 小
30         int pos = lim;
31         for (int i = p - 1; i >= 0; i--) {
32             if (d[pos - (1 << i)] >= k) {
33                 pos -= (1 << i);
34             } else {
35                 k -= d[pos - (1 << i)];
36             }
37         }
38         return pos;
```



```
39 |     }  
40 | };
```

II. 可持久化线段树

可持久化线段树封装模板（update 和 query 的具体功能需根据题目重新实现）

```
1 struct Persistable_SegT {
2     struct Node {
3         int ls, rs;
4         ll sum; int tot;
5     };
6     int cnt_id;
7     Node tr[MAXN << 6];
8
9     #define mid ((l + r) >> 1)
10    #define ls(x) tr[x].ls
11    #define rs(x) tr[x].rs
12    #define sum(x) tr[x].sum
13    #define tot(x) tr[x].tot
14
15    void push_up(int rt) {
16        sum(rt) = sum(ls(rt)) + sum(rs(rt));
17        tot(rt) = tot(ls(rt)) + tot(rs(rt));
18    }
19
20    void update(int &rt, int pre, int l, int r, int idx, ll v) {
21        rt = ++cnt_id;
22        ls(rt) = ls(pre);
23        rs(rt) = rs(pre);
24        if(l == r) {
25            tr[rt].sum = tr[pre].sum + v;
26            tr[rt].tot = tr[pre].tot + 1;
27            return ;
28        }
29        if (idx <= mid) {
30            update(ls(rt), ls(pre), l, mid, idx, v);
31        }
32        else {
33            update(rs(rt), rs(pre), mid + 1, r, idx, v);
34        }
35        push_up(rt);
36    }
37    ll query(int lrt, int rrt, int l, int r, int k, ll *lisan) {
38        if (k <= 0) {
39            return 0;
40        }
41        if (l == r && tot(rrt) - tot(lrt) > k) {
42            return lisan[1] * k;
43        }
```

```

44         if (tot(rrt) - tot(lrt) <= k) {
45             return sum(rrt) - sum(lrt);
46         }
47         ll res = 0;
48         int cnt = tot(rs(rrt)) - tot(rs(lrt));
49         res += query(rs(lrt), rs(rrt), mid + 1, r, k, lisan);
50         res += query(ls(lrt), ls(rrt), l, mid, k - cnt, lisan);
51         return res;
52     }
53
54     #undef ls
55     #undef rs
56     #undef sum
57     #undef mid
58
59 } SegT;

```

III. 线段树合并

线段树合并封装模板

```
1 struct SegT {
2     struct Node {
3         int ls, rs, tot;
4     } tr[MAXN << 6];
5     int cnt_id = 0;
6
7     #define mid ((l + r) >> 1)
8     #define ls(x) tr[x].ls
9     #define rs(x) tr[x].rs
10
11 void push_up(int rt) {
12     tr[rt].tot = tr[ls(rt)].tot + tr[rs(rt)].tot;
13 }
14 void upd(int &rt, int l, int r, int idx) {
15     if (!rt) {
16         rt = ++cnt_id;
17     }
18     if (l == r) {
19         tr[rt].tot++;
20         return ;
21     }
22     if (idx <= mid) {
23         upd(ls(rt), l, mid, idx);
24     } else {
25         upd(rs(rt), mid + 1, r, idx);
26     }
27     push_up(rt);
28 }
29 int query(int rt, int l, int r, int sp, int ep) {
30     if (sp <= l && r <= ep) {
31         return tr[rt].tot;
32     }
33     int res = 0;
34     if (sp <= mid) {
35         res += query(ls(rt), l, mid, sp, ep);
36     }
37     if (ep > mid) {
38         res += query(rs(rt), mid + 1, r, sp, ep);
39     }
40     return res;
41 }
42 int merge(int x, int y) {
43     if (!x || !y) {
```

```
44         return x ^ y;
45     }
46     tr[x].tot += tr[y].tot;
47     ls(x) = merge(ls(x), ls(y));
48     rs(x) = merge(rs(x), rs(y));
49     return x;
50 }
51
52 #undef mid
53 #undef ls
54 #undef rs
55 } Tr;
```

IV. 李超线段树

李超线段树封装模板，给定一堆直线，求某个 x 处所有直线的 y 的最大值。

```
1 struct SegT_LiChao {
2     static const int MAXN = 1e5 + 10;
3     static const int N = 39989 + 10;
4     const double eps = 1e-9;
5     int fcmp(double x, double y) {
6         if (x - y > eps) return 1;
7         if (y - x > eps) return -1;
8         return 0;
9     }
10    void chk_mx(pdi &x, pdi y) {
11        if (y.fi - x.fi > eps || (fcmp(x.fi, y.fi) == 0 && x.se > y.se)) {
12            x = y;
13        }
14    }
15
16    struct Line {
17        double k, b;
18        int L, R;
19        Line() { k = 0, b = -1; }
20        Line(double k, double b, int L, int R): k(k), b(b), L(L), R(R) {}
21    };
22    int cntl = 0;
23    Line p[MAXN];
24    void addl(int x1, double y1, int x2, double y2) {
25        if (x1 > x2) {
26            swap(x1, x2);
27            swap(y1, y2);
28        }
29        if (x1 == x2) {
30            p[++cntl] = {0, (double)max(y1, y2), x1, x2};
31        }
32        else {
33            double k = 1.0 * (y2 - y1) / (x2 - x1), b = y1 - k * x1;
34            p[++cntl] = {k, b, x1, x2};
35        }
36        modify(1, 1, N, cntl);
37    }
38    pdi query(int idx) {
39        return query(1, 1, N, idx);
40    }
41    double calc(int u, int x) {
42        return p[u].k * x + p[u].b;
43    }
```

```

44
45     #define lson (rt << 1)
46     #define rson (rt << 1 | 1)
47     #define mid ((l + r) >> 1)
48
49     int tr[N << 2];
50     void upd(int rt, int l, int r, int u) {
51         int &v = tr[rt];
52         int op = fcmp(calc(u, mid), calc(v, mid));
53         if (op == 1 || (op == 0 && u < v)) {
54             swap(u, v);
55         }
56         int opl = fcmp(calc(u, l), calc(v, l));
57         int opr = fcmp(calc(u, r), calc(v, r));
58         if (opl == 1 || (opl == 0 && u < v)) upd(lson, l, mid, u);
59         if (opr == 1 || (opr == 0 && u < v)) upd(rson, mid + 1, r, u);
60     }
61     void modify(int rt, int l, int r, int u) {
62         if (p[u].L <= l && r <= p[u].R) {
63             upd(rt, l, r, u);
64             return ;
65         }
66         if (p[u].L <= mid) modify(lson, l, mid, u);
67         if (p[u].R > mid) modify(rson, mid + 1, r, u);
68     }
69     pdi query(int rt, int l, int r, int idx) {
70         if (l == r) {
71             return mkp(calc(tr[rt], idx), tr[rt]);
72         }
73         pdi res = mkp(calc(tr[rt], idx), tr[rt]);
74         if (idx <= mid) chk_mx(res, query(lson, l, mid, idx));
75         else chk_mx(res, query(rson, mid + 1, r, idx));
76         return res;
77     }
78
79     #undef lson
80     #undef rson
81     #undef mid
82 } Tr;

```

V. 动态开点线段树求解线段覆盖问题

```
1 struct SegT {
2     int cnt_id;
3     struct Node {
4         int ls, rs;
5         bool flg;
6         int sum;
7     } tr[MAXN * 100];
8
9     #define mid ((l + r) >> 1)
10    void clear() {
11        for (int i = 0; i <= cnt_id; i++) {
12            tr[i].ls = tr[i].rs = tr[i].sum = 0;
13            tr[i].flg = false;
14        }
15        cnt_id = 0;
16    }
17    void push_up(int rt, int l, int r) {
18        if (tr[rt].flg) {
19            tr[rt].sum = r - l + 1;
20        } else {
21            tr[rt].sum = tr[tr[rt].ls].sum + tr[tr[rt].rs].sum;
22        }
23    }
24    void upd(int &rt, int l, int r, int sp, int ep) {
25        if (!rt) {
26            rt = ++cnt_id;
27            assert(cnt_id < MAXN * 100);
28        }
29        if (sp <= l && r <= ep) {
30            tr[rt].flg = true;
31            push_up(rt, l, r);
32            return ;
33        }
34        if (sp <= mid) {
35            upd(tr[rt].ls, l, mid, sp, ep);
36        }
37        if (ep > mid) {
38            upd(tr[rt].rs, mid + 1, r, sp, ep);
39        }
40        push_up(rt, l, r);
41    }
42    #undef mid
43 } Tr;
```


VI. Treap

Fhq_treap 封装模板

```
1 mt19937 mrnd(std::chrono::steady_clock::now().time_since_epoch().count());
2 int rnd(int l, int r) {
3     return mrnd() % (r - l + 1) + l;
4 }
5 double rnd01() {
6     return mrnd() * 1.0 / UINT32_MAX;
7 }
8
9 namespace Fhq_Treap {
10     const int MAXN = 5e5 + 10;
11
12     struct Node {
13         int lson, rson;
14         int pri, sz;
15         ll val;
16     };
17
18     int cnt_id;
19     Node tree[MAXN << 4];
20     queue<int> rub;
21     #define ls(x) tree[x].lson
22     #define rs(x) tree[x].rson
23
24     struct Tree {
25         int rt = 0;
26         void clear(int x) {
27             tree[x] = Node();
28             if (x != 0) {
29                 rub.push(x);
30             }
31         }
32         void all_clear() {
33             rt = 0;
34             auto vec = iter_id();
35             for (auto x : vec) {
36                 clear(x);
37             }
38         }
39         vector<int> iter_id() { // 中序遍历
40             vector<int> res;
41             auto dfs = [&](auto &&self, int x) -> void {
42                 if (!x) {
43                     return ;
```

```

44         }
45         self(self, ls(x));
46         res.push_back(x);
47         self(self, rs(x));
48     };
49     dfs(dfs, rt);
50     return res;
51 }
52 vector<ll> iter_val() {
53     vector<ll> res;
54     auto dfs = [&](auto &&self, int x) -> void {
55         if (!x) {
56             return ;
57         }
58         self(self, ls(x));
59         res.push_back(tree[x].val);
60         self(self, rs(x));
61     };
62     dfs(dfs, rt);
63     return res;
64 }
65 int newp(ll val) {
66     int cur;
67     if (!rub.empty()) {
68         cur = rub.front();
69         rub.pop();
70     } else {
71         cur = ++cnt_id;
72     }
73     tree[cur].val = val;
74     tree[cur].pri = rnd(1, 114514);
75     tree[cur].sz = 1;
76     return cur;
77 }
78 void push_up(int x) {
79     tree[x].sz = tree[ls(x)].sz + tree[rs(x)].sz + 1;
80 }
81 void split(int x, int &l, int &r, ll val) {
82     if (!x) {
83         l = r = 0;
84         return ;
85     }
86     if (tree[x].val <= val) {
87         l = x;
88         split(rs(x), rs(l), r, val);
89     } else {
90         r = x;
91         split(ls(x), l, ls(r), val);

```

```

92         }
93     push_up(x);
94 }
95 int merge(int x, int y) {
96     if (!x || !y) {
97         return x ^ y;
98     }
99     if (tree[x].pri <= tree[y].pri) {
100         rs(x) = merge(rs(x), y);
101         return push_up(x), x;
102     } else {
103         ls(y) = merge(x, ls(y));
104         return push_up(y), y;
105     }
106 }
107 void insert(ll val) {
108     int l_rt, r_rt;
109     split(rt, l_rt, r_rt, val);
110     rt = merge(merge(l_rt, newp(val)), r_rt);
111 }
112 void del(ll val) {
113     int l_rt, r_rt, mid_rt;
114     split(rt, l_rt, r_rt, val);
115     split(l_rt, l_rt, mid_rt, val - 1);
116     mid_rt = merge(ls(mid_rt), rs(mid_rt));
117     rt = merge(merge(l_rt, mid_rt), r_rt);
118 }
119 int size() {
120     return tree[rt].sz;
121 }
122 int greater_equal_cnt(ll val) {
123     int l_rt, r_rt;
124     split(rt, l_rt, r_rt, val - 1);
125     int res = tree[r_rt].sz;
126     rt = merge(l_rt, r_rt);
127     return res;
128 }
129 ll kth_mx(int k) {
130     int x = rt;
131     while (true) {
132         if (tree[rs(x)].sz + 1 < k) {
133             k -= tree[rs(x)].sz + 1;
134             x = ls(x);
135         }
136         else if (tree[rs(x)].sz + 1 > k) {
137             x = rs(x);
138         }
139         else {

```

```
140         return tree[x].val;
141     }
142 }
143 }
144 };
145 }
146 using Fhq_Treap::Tree;
```

文艺平衡树（带区间翻转标记的 fhq_treap）代码

```
1  #include <iostream>
2  #include <cstdio>
3  #include <cstring>
4  #include <algorithm>
5  #include <cmath>
6  #include <cstdlib>
7  #include <ctime>
8
9  using namespace std;
10
11  const int MAXN=1e5+10;
12
13  struct Fhq_treap
14  {
15      #define lson(x) tree[x].ls
16      #define rson(x) tree[x].rs
17
18      struct Node
19      {
20          int ls,rs,sz;
21          int pri,val;
22          bool rev;
23      };
24
25      int root,cnt_id;
26      Node tree[MAXN];
27
28      inline int newp(int val)
29      {
30          tree[++cnt_id].val=val;
31          tree[cnt_id].sz=1;
32          tree[cnt_id].pri=rand();
33          return cnt_id;
34      }
35
36      inline void push_up(int x){
37          tree[x].sz=tree[lson(x)].sz+tree[rson(x)].sz+1; }
38      inline void push_down(int x)
39      {
40          if(tree[x].rev)
41          {
42              swap(lson(x),rson(x));
43              tree[lson(x)].rev^=1;
44              tree[rson(x)].rev^=1;
45              tree[x].rev=false;
```

```

45     }
46 }
47
48 inline void split(int x,int &l,int &r,int val)
49 {
50     if(!x) return l=r=0,(void)0;
51
52     push_down(x);
53     if(tree[lson(x)].sz+1<=val) l=x,split(rson(x),rson(x),r,val-
tree[lson(x)].sz-1);
54     else r=x,split(lson(x),l,lson(x),val);
55     push_up(x);
56 }
57
58 inline int merge(int l,int r)
59 {
60     if(!l || !r) return l^r;
61
62     if(tree[l].pri<tree[r].pri)
63     {
64         push_down(l);
65         rson(l)=merge(rson(l),r);
66         return push_up(l),l;
67     }
68     else
69     {
70         push_down(r);
71         lson(r)=merge(l,lson(r));
72         return push_up(r),r;
73     }
74 }
75
76 inline void insert(int val)
77 {
78     int l_rt,r_rt;
79     split(root,l_rt,r_rt,val-1);
80     root=merge(merge(l_rt,newp(val)),r_rt);
81 }
82
83 inline void update(int l,int r)
84 {
85     int l_rt,r_rt,mid_rt;
86     split(root,l_rt,r_rt,l-1);
87     split(r_rt,mid_rt,r_rt,r-l+1);
88     tree[mid_rt].rev^=1;
89     root=merge(l_rt,merge(mid_rt,r_rt));
90 }
91

```

```

92     inline void print(int x)
93     {
94         if(!x) return ;
95         push_down(x);
96
97         print(lson(x));
98         printf("%d ", tree[x].val);
99         print(rson(x));
100    }
101 };
102
103 int n,m;
104 Fhq_treap Treap;
105
106 int main()
107 {
108     scanf("%d%d",&n,&m);
109     for(int i=1;i<=n;i++) Treap.insert(i);
110
111     while(m--)
112     {
113         int l,r;
114         scanf("%d%d",&l,&r);
115         Treap.update(l,r);
116     }
117     Treap.print(Treap.root);
118
119     return 0;
120 }

```

VII. 笛卡尔树的线性构建

```
1 struct CartesianTr {
2     int n, rt;
3     vector<int> ls, rs;
4     vector<int> L, R;
5     vector<ll> val;
6     void build(int n, const vector<ll>& a) {
7         this->n = n;
8         ls.resize(n + 1); L.resize(n + 1);
9         rs.resize(n + 1); R.resize(n + 1);
10        val.resize(n + 1);
11
12        int tp = 0;
13        vector st(n + 1, 0);
14        for (int i = 1; i <= n; i++) {    // heap 是小根堆
15            int p = tp;
16            val[i] = a[i];
17            while (p && val[st[p]] > val[i]) {
18                p--;
19            }
20            if (p) {
21                rs[st[p]] = i;
22            }
23            if (p < tp) {
24                ls[i] = st[p + 1];
25            }
26            st[++p] = i;
27            tp = p;
28        }
29        rt = st[1];
30
31        auto dfs = [&](auto &&self, int u, int cl, int cr) -> void {
32            if (!u) {
33                return ;
34            }
35            L[u] = cl;
36            R[u] = cr;
37            self(self, ls[u], cl, u - 1);
38            self(self, rs[u], u + 1, cr);
39        };
40        dfs(dfs, rt, 1, n);
41    }
42};
```


Ⅷ. `__gnu_pbds::tree` —— C++扩展库中的平衡树使用方法

```
1  __gnu_pbds::tree<std::pair<int, int>, __gnu_pbds::null_type,  
2                      std::less<std::pair<int, int>>, __gnu_pbds::rb_tree_tag,  
3                      __gnu_pbds::tree_order_statistics_node_update>  
4      trr;
```

- `insert(x)`: 向树中插入一个元素 **x**, 返回 `std::pair<point_iterator, bool>`。
- `erase(x)`: 从树中删除一个元素/迭代器 **x**, 返回一个 `bool` 表明是否删除成功。
- `order_of_key(x)`: 返回 **x** 以 `Cmp_Fn` 比较的排名 (例如使用 `std::less<int>`, 则返回严格小于 **x** 的元素的个数)。
- `find_by_order(x)`: 返回 `Cmp_Fn` 比较的排名所对应元素的迭代器 (注意: 排名编号从 **0** 开始)。
- `lower_bound(x)`: 以 `Cmp_Fn` 比较做 `lower_bound`, 返回迭代器。
- `upper_bound(x)`: 以 `Cmp_Fn` 比较做 `upper_bound`, 返回迭代器。
- `join(x)`: 将 **x** 树并入当前树, 前提是两棵树的类型一样, **x** 树被删除。
- `split(x,b)`: 以 `Cmp_Fn` 比较, 小于等于 **x** 的属于当前树, 其余的属于 **b** 树。
- `empty()`: 返回是否为空。
- `size()`: 返回大小。

IX. 并查集

```
1 struct Dsu {
2     vector<int> fa;
3     void init(int n) {
4         fa.resize(n + 1, 0);
5         for (int i = 1; i <= n; i++) {
6             fa[i] = i;
7         }
8     }
9     int fifa(int x) {
10         if (fa[x] == x) {
11             return x;
12         }
13         return fa[x] = fifa(fa[x]);
14     }
15     void merge(int x, int y) {
16         x = fifa(x);
17         y = fifa(y);
18         if (x != y) {
19             fa[x] = y;
20         }
21     }
22 };
```

二. 分治相关

I. cdq 分治

利用 cdq 分治离线解决三维偏序问题。

```
1 BIT<int> tr;
2 tr.init(m);
3 auto cdq = [&](auto &&self, int l, int r) -> void {
4     if (l == r) {
5         return ;
6     }
7
8     int mid = (l + r) >> 1;
9     self(self, l, mid);
10    self(self, mid + 1, r);
11
12    int i = l, j = mid + 1, p = 0;
13    vector tmp(r - l + 1, Node());
14    while (i <= mid && j <= r) {
15        if (a[i].y <= a[j].y) {
16            tr.upd(a[i].z, a[i].tot);
17            tmp[p++] = a[i++];
18        } else {
19            a[j].ans += tr.query(a[j].z);
20            tmp[p++] = a[j++];
21        }
22    }
23    while (i <= mid) {
24        tr.upd(a[i].z, a[i].tot);
25        tmp[p++] = a[i++];
26    }
27    while (j <= r) {
28        a[j].ans += tr.query(a[j].z);
29        tmp[p++] = a[j++];
30    }
31
32    for (int k = l; k <= mid; k++) {
33        tr.upd(a[k].z, -a[k].tot);
34    }
35    for (int k = l; k <= r; k++) {
36        a[k] = tmp[k - l];
37    }
38 };
```

II. 整体二分

在询问答案具有单调性时，二分的思路是， $\text{Solve}(l, r)$ 表示我二分此问题的答案时，已经知晓了 $ans \in [l, r]$ ，需要做进一步的递归。此时如果有一种手段 $\text{check}(x)$ 表示判断 $ans \geq x$ 是否是合法的，这个问题就得到了解决：令 $mid = (l + r)/2$ ， $\text{check}(mid)$ 后执行 $\text{Solve}(l, mid)$ 或 $\text{Solve}(mid, r)$ 之一，直到边界情况 ($l = r$)。

但面对多个询问，这种方式效率低下。考虑一种方式： $\text{Solve}(l, r, Q)$ 表示同时解决集合 Q 内的所有询问（已知答案的值域在 $[l, r]$ ），利用数据结构对 mid 进行复杂度与 $|Q|, [l, r]$ 有关的处理，而后对集合内所有询问执行 $\text{check}(mid)$ ，将所有询问分为两组 Q_1, Q_2 分别表示 $ans < mid$ 的与 $\geq mid$ 的，再调用 $\text{Solve}(l, mid, Q_1)$ 与 $\text{Solve}(mid, r, Q_2)$ 递归解决下去。

例 0 (P3834)

题意：给一个长度为 n 的序列， q 次区间 $[L, R]$ 查询 k 小。
保证 $n, q \leq 10^5$ ，要求线性空间。

Sol：先定义 $\text{Solve}(l, r, Q)$ 表示已知集合 Q 内的询问的答案在 $[l, r]$ 中，考虑如何分治下去，也就是判断 Q 内所有询问的答案与 mid 的大小关系。

直观的想法是，将序列中的 $> mid$ 的数看做 1，否则看做 0，询问 (L, R, k) 即判断 $[L, R]$ 的区间和与 k 的大小关系。最为朴素的实现是直接在原序列中枚举每个元素，做一个前缀和，时间复杂度是无法承受的。

观察性质，我们现在要解决的是形如「区间内 $< x$ 的数有多少个」的问题，贡献是可加的，不妨将其看做值域在 $(-\infty, l)$ 与 $[l, mid]$ 两部分中的数的和，而前者已经在之前的分治中求出（不然不会分治到当前区间），后者可以暴力枚举序列中 $[l, mid]$ 中的数做单点加。利用树状数组维护区间和，就可以对每个询问 $O(\log n)$ 判断。

分析这样做的复杂度：一个重要的结论是，分治深度为 $O(\log n)$ 级别。对于某个序列中的元素 x ，最多有 $O(\log n)$ 个 Solve 的区间包含之，也就是最多做 $O(\log n)$ 次对于这个数的单点加。而对于某个询问，最多只会被判断 $O(\log n)$ 次，也最多只会被“划分” $O(\log n)$ 次，故这种做法时间复杂度为 $O((n + q) \log^2 n)$ ，空间复杂度 $O(n + q)$ 。

同样这题有 $O(n \log n)$ 的整体二分做法，在后面会提到。

例 0.5 (P2617)

题意：例 0，带单点修改（将 pos 上的数修改为 x ）。
数据范围同例 0。

Sol：带上了时间轴，看起来不是静态二分能处理的问题，实则容易。

在分治时，我们分的是值域，对询问 check 的时间顺序是没有要求的，此时整体二分的优越性得以体现：可以将询问按照时间轴排序。如果我们能把修改也放到这个时间轴当中并在线地按照时间轴执行，那么处理询问时的答案就是真实的了。修改的权值如何定义？

一次修改 (pos, x) ，可以看做在 pos 位置删除了一个 a_{pos} ，并添加了一个 x ，通过这种拆成一删一插的方式，使得拆完之后的两个修改拥有了值域上的属性，可以放进去二分。具体而言： $\text{Solve}(l, r, Q)$ 中的 Q 此时表示的修改与询问的集合，并已经按照时间排序。先让树状数组呈现「仅 $[l, mid]$ 中的数为 1，其他为 0」的状态，再按时间顺序执行 Q 内的询问与修改。询问按照问出的值划分，修改的权值是确定的，直接划分。时间复杂度依然是 $O((n + q) \log^2 n)$ 。

整体二分维护不可加贡献

现在回顾例 0 中的一句话：

贡献是可加的，不妨将其看做值域在 $(-\infty, l)$ 与 $[l, mid]$ 两部分中的数的和，而前者已经在之前的分治中求出...

这很丑陋，因为贡献不需要可加啊？我们可以这样子操作：每次 $\text{Solve}(l, r, Q)$ 时，先执行 $[l, mid]$ 的修改，解决 Q 的询问并将 Q 分为两半，然后**不清空**，向右 $\text{Solve}(mid, r, Q_2)$ ，再**撤销掉** $[l, mid]$ 的操作，执行 $\text{Solve}(l, mid, Q_1)$ 。

这种做法的不同之处在于， Solve 前保证了左端点之前的修改已经被执行，进而直接获得「执行了 $(-\infty, mid)$ 内的修改」这个状态，做完后撤销回去。好处就是，询问的答案不需要分多次可加的贡献得到，而是可以直接查询，这样就可以维护一类并不是很好拆分的贡献，比如**图连通性**。

例 3 (CF603E)

题意：给定一张 n 个点的无向图，初始没有边。依次加入 m 条带权的边，每次加入后询问：是否存在一个边集使得图中所有点的度数均为奇数。若存在，则输出边集中的最大边权的最小值。保证 $n \leq 10^5, m \leq 3 \times 10^5$ 。

Sol：首先是一波人类智慧操作，结论是：存在合法边集当且仅当所有连通块大小均为偶数。

- 必要性：连通块大小为奇数时若存在方案，则保留合法边集后此连通块度数之和为奇数，矛盾。
- 充分性：每个联通块内仅保留 DFS 树，自底向上 DP，必然可以构造方案。

然后考虑无修改的情况：连通块大小均为偶数时，再添加一些边后依然满足条件，所以按边权从小到大排序后，有用的边一定是一个前缀，并且具有单调性。而此题「带修改，多询问」，并且询问的答案有很好的性质：单调不减。自然想到整体二分。

我们用数据结构（可撤销并查集）维护连通块，观察到答案序列单调，考虑使用上文求解单调序列的方式： $\text{Solve}(l, r, L, R)$ 表示已知 $ans_{[l, r]}$ 在 $[L, R]$ 之中，并且权值 $< L$ 且时间 $< l$ 的边已经加在并查集中，我需要进一步分治下去。令 $mid = \frac{l+r}{2}$ ，考虑求出 ans_{mid} 的值 p 后，分别分治做 $\text{Solve}(l, mid - 1, L, p)$ ， $\text{Solve}(mid + 1, r, p, R)$ 。

怎样才能求得 ans_{mid} ？考虑最为朴素的做法：将时间 $\leq mid$ 的边按照权值从小到大加入到并查集中，第一个连通块大小均为偶数的时刻即为 ans_{mid} 。而此时已经保证了权值 $< L$ 且时间 $< l$ 的边已经加在并查集中，只需要按顺序加入：

1. 时间 $\in [l, mid]$ 且权值 $< L$ 的边。这部分边不影响答案，先直接加入。
2. 权值 $\in [L, R]$ 且时间 $\leq mid$ 的边，加到图合法为止。

图第一次合法时得出了 ans_{mid} 的答案 p ，此时我们撤销到这次 Solve 之前，加入上文 1 中的边并递归 $\text{Solve}(mid + 1, r, L, p)$ ，然后撤销之，加入 2 中的边并递归 $\text{Solve}(l, mid, p, R)$ 。每次 Solve 的复杂度是 $O((R - L + r - l) \log n)$ ，分治层数 $O(\log n)$ ，故总复杂度 $O(m \log^2 n)$ ，空间复杂度 $O(n + m)$ 。

三. 其他数据结构

I. ST 表 (求解区间 RMQ)

```
1  vector rmq(n + 1, vector(21, 0));
2  for (int i = 1; i <= n; i++) {
3      rmq[i][0] = sum[i];
4  }
5  for (int j = 1; j <= 20; j++) {
6      for (int i = 1; i + (1 << (j - 1)) <= n; i++) {
7          rmq[i][j] = max(rmq[i][j - 1], rmq[i + (1 << (j - 1))][j - 1]);
8      }
9  }
10 vector lg2(n + 1, 0);
11 for (int i = 2; i <= n; i++) {
12     lg2[i] = lg2[i >> 1] + 1;
13 }
14 auto qmax = [&](int l, int r) -> int {
15     int s = lg2[r - l + 1];
16     return max(rmq[l][s], rmq[r - (1 << s) + 1][s]);
17 };
```

图论部分

一. 最短路与最小生成树进阶

I. 同余最短路

通过同余构造某些状态，状态之间的关系类似于两点之间的带权有向边。

那么可以以此建图，将某些问题转化为最短路问题，再使用具有优秀时间复杂度的算法求解。

【例题】P3403 跳楼机

① Note

有 h 个格子，初始在 1 号格子，每次可以

- 向右移动 x 个格子
- 向右移动 y 个格子
- 向右移动 z 个格子
- 回到第一个格子

问能到达格子的总数量。

首先可以将 h 减去 1，同时起始楼层设为 0。

设 d_i 为从 1 开始能够到达的最低的 $\text{mod } x = i$ 的楼层。

则有 $i \xrightarrow{y} (i + y) \bmod x$ 和 $i \xrightarrow{z} (i + z) \bmod x$ 。

像这样建图后， d_i 就相当于 $0 \rightarrow i$ 的最短路，Dijkstra 即可。

最后统计时，对于 $d_i \leq h$ ，有贡献 $\lfloor \frac{h-d_i}{x} \rfloor + 1$ 。

总时间复杂度 $\mathcal{O}(n \log n)$ 。

```
1  const int N = 1e5 + 7;
2  const ll inf = (1ull << 63) - 1;
3  ll h, d[N], ans;
4  int x, y, z, v[N];
5  vector<pi> e[N];
6  pq<pair<ll, int>> q;
7
8  int main() {
9      rd(h), --h, rd(x), rd(y), rd(z);
10     for (int i = 0; i < x; i++) e[i].pb(mp((i + y) % x, y)), e[i].pb(mp((i + z) % x, z)), d[i] = inf;
```



```

11     d[0] = 0, q.push(mp(0, 0));
12     while (q.size()) {
13         int x = q.top().se;
14         q.pop();
15         if (v[x]) continue;
16         v[x] = 1;
17         for (ui i = 0; i < e[x].size(); i++) {
18             int y = e[x][i].fi, z = e[x][i].se;
19             if (d[y] > d[x] + z) d[y] = d[x] + z, q.push(mp(-d[y], y));
20         }
21     }
22     for (int i = 0; i < x; i++)
23         if (h >= d[i]) ans += (h - d[i]) / x + 1;
24     print(ans);
25     return 0;
26 }

```

【例题】P2371[国家集训队]墨墨的等式

上一题的扩展，不再是只能向右走 x, y, z 格，变成能向右走 $a_1, a_2, \dots, a_n$ 格。

```

1  const int N = 5e5 + 7;
2  const ll inf = 1e18;
3  ll l, r, d[N], ans;
4  int n, x, v[N];
5  vector<pi> e[N];
6  pq<pair<ll, int>> q;
7
8  int main() {
9      rd(n), rd(l), --l, rd(r), rd(x);
10     for (int i = 1; i < x; i++) d[i] = inf;
11     for (int i = 1, y; i < n; i++) {
12         rd(y);
13         for (int i = 0; i < x; i++)
14             e[i].pb(mp((i + y) % x, y));
15     }
16     q.push(mp(0, 0));
17     while (q.size()) {
18         int x = q.top().se;
19         q.pop();
20         if (v[x]) continue;
21         v[x] = 1;
22         for (ui i = 0; i < e[x].size(); i++) {
23             int y = e[x][i].fi, z = e[x][i].se;
24             if (d[y] > d[x] + z) d[y] = d[x] + z, q.push(mp(-d[y], y));
25         }
26     }
27     for (int i = 0; i < x; i++) {

```

```
28         if (r >= d[i]) ans += (r - d[i]) / x + 1;
29         if (l >= d[i]) ans -= (l - d[i]) / x + 1;
30     }
31     print(ans);
32     return 0;
33 }
```

II. kruskal 重构树

Kruskal 重构树的定义（构造方式）：

- 一开始，各个结点都在一个以自己为根的连通块中。
- 执行 Kruskal 算法，重复以下步骤。
 1. 把即将加入生成树边集的边，看做一个新点 x ， x 的两儿子分别是 u 所在的连通块对应的根和 v 所在的连通块对应的根， x 的点权是该边的边权。
 2. 合并 u, v ，这个合并后的连通块的根是 x 。

性质：

- Kruskal 重构树是一个二叉树。
- Kruskal 重构树的叶子没有点权，其他点都有点权。
- Kruskal 重构树是完全二叉树（除叶结点外，每个结点都有两个儿子）。
- 原图中的两点 u, v 在 Kruskal 重构树上的 LCA 的点权是原图中 u 到 v 的所有路径中最大边的最小值。

二. 连通性相关

I. tarjan 算法求强连通分量并缩点

```
1  int tp = 0, cnt_id = 0, cnt_clr = 0;
2  vector st(n + 1, 0), inst(n + 1, 0), sz(n + 1, 0);
3  vector dfn(n + 1, 0), low(n + 1, 0), clr(n + 1, 0);
4  vector val(n + 1, 0);
5  auto tarjan = [&](auto&& self, int u) -> void {
6      st[++tp] = u;
7      inst[u] = true;
8      dfn[u] = low[u] = ++cnt_id;
9      for (auto v : G[u]) {
10         if (!dfn[v]) {           // 树边
11             self(self, v);
12             low[u] = min(low[u], low[v]);
13         }
14         else if (inst[v]) {       // 返祖边
15             low[u] = min(low[u], dfn[v]);
16         }
17         // 横叉边不需要处理
18     }
19
20     if (dfn[u] == low[u]) {
21         cnt_clr++;
22         do {
23             sz[cnt_clr]++;
24             clr[st[tp]] = cnt_clr;
25             val[cnt_clr] += a[st[tp]];
26             inst[st[tp]] = false;
27         } while (st[tp--] != u);
28     }
29 };
```

II. tarjan 算法求边双连通分量并缩点

```
1  const int MAXN = 5e5 + 10;
2
3  int n, m;
4  vector<int> G[MAXN];
5
6  int dfn[MAXN], low[MAXN];
7  int st[MAXN], tp, cnt_id;
8  bool inst[MAXN];
9  vector<vector<int>> ans;
10
11 void tarjan(int u, int fa) {
12
13     dfn[u] = low[u] = ++cnt_id;
14     st[++tp] = u; inst[u] = true;
15
16     int cnt = 0;
17
18     for (auto v : G[u]) {
19         if (v == fa) {
20             cnt++;
21             if (cnt == 1) continue;
22         }
23         if (!dfn[v]) {
24             tarjan(v, u);
25             low[u] = min(low[u], low[v]);
26         }
27         else if (inst[v]) {
28             low[u] = min(low[u], dfn[v]);
29         }
30     }
31
32     if (dfn[u] == low[u]) {
33         vector<int> cur;
34         do {
35             inst[st[tp]] = false;
36             cur.push_back(st[tp]);
37         } while (st[tp--] != u);
38         ans.push_back(cur);
39     }
40 }
41
42 int main() {
43     cin >> n >> m;
44     for (int i = 1; i <= m; i++) {
45         int u, v;
```

```

46         cin >> u >> v;
47         G[u].push_back(v);
48         G[v].push_back(u);
49     }
50
51     for (int i = 1; i <= n; i++)
52         if (!dfn[i]) {
53             tarjan(i, 0);
54         }
55
56     printf("%d\n", SZ(ans));
57     for (auto vec : ans) {
58         printf("%d ", SZ(vec));
59         for (auto x : vec) {
60             printf("%d ", x);
61         }
62         puts("");
63     }
64
65     return 0;
66 }

```

III. tarjan 算法求点双连通分量并建立圆方树

例题 SDOI2018 战略游戏，建出圆方树后，求两点路径上的圆点数量。

```
1  const int MAXN = 1e5 + 10;
2
3  int n, m;
4  vector<int> G[MAXN], trG[MAXN << 1];
5  // trG : 存圆方树
6
7  int cnt_id, idx_squ;
8  // cnt_id : 原图的 dfs 序计数器
9  // idx_squ : 圆方树上方点标号的计数器
10 int dfn[MAXN << 1], low[MAXN];
11 int st[MAXN], tp;
12 void tarjan(int u) {
13     dfn[u] = low[u] = ++cnt_id;
14     st[++tp] = u;
15     for (auto v : G[u]) {
16         if (!dfn[v]) {
17             tarjan(v);
18             low[u] = min(low[u], low[v]);
19
20             if (dfn[u] == low[v]) {
21                 idx_squ++;
22                 for (int x = 0; x != v; tp--) {
23                     x = st[tp];
24                     trG[idx_squ].push_back(x);
25                     trG[x].push_back(idx_squ);
26                 }
27                 trG[idx_squ].push_back(u);
28                 trG[u].push_back(idx_squ);
29             }
30         } else {
31             low[u] = min(low[u], dfn[v]);
32         }
33     }
34 }
35
36 // 注意对于圆方树进行处理的数组都需要开两倍空间
37 int dep[MAXN << 1], dis[MAXN << 1];
38 int fa[MAXN << 1][20];
39 void dfs_init(int u, int fz, int d) {
40     dfn[u] = ++cnt_id;
41     fa[u][0] = fz;
42     dep[u] = d;
43     dis[u] = dis[fz] + (u <= n);
```

```

44     for (auto v : trG[u]) {
45         if (v == fz) {
46             continue;
47         }
48         dfs_init(v, u, d + 1);
49     }
50 }
51
52 int qlca(int x, int y) {
53     if (dep[x] < dep[y]) {
54         swap(x, y);
55     }
56     int t = dep[x] - dep[y];
57     for (int i = 19; i >= 0; i--) {
58         if ((t >> i) & 1) {
59             x = fa[x][i];
60         }
61     }
62     if (x == y) {
63         return x;
64     }
65     for (int i = 19; i >= 0; i--) {
66         if (fa[x][i] != fa[y][i]) {
67             x = fa[x][i];
68             y = fa[y][i];
69         }
70     }
71     return fa[x][0];
72 }
73
74 void init() {
75     for (int i = 1; i <= n; i++) {
76         G[i].clear();
77     }
78     for (int i = 1; i <= (n << 1); i++) {
79         dfn[i] = 0;
80         trG[i].clear();
81     }
82 }
83
84 void solve() {
85     cin >> n >> m;
86     init();
87
88     for (int i = 1; i <= m; i++) {
89         int u, v;
90         cin >> u >> v;
91         G[u].push_back(v);

```



```

92         G[v].push_back(u);
93     }
94
95     tp=0; idx_squ = n;
96     cnt_id = 0; tarjan(1);
97     cnt_id = 0; dfs_init(1, 0, 0);
98
99     for (int j = 1; j <= 19; j++) {
100         for (int i = 1; i <= idx_squ; i++) {
101             fa[i][j] = fa[fa[i][j - 1]][j - 1];
102         }
103     }
104
105     int q;
106     cin >> q;
107     while (q--) {
108         int s;
109         cin >> s;
110         vector p(s + 1, 0);
111         for (int i = 1; i <= s; i++) {
112             cin >> p[i];
113         }
114         sort(p.begin() + 1, p.begin() + s + 1, [](auto x, auto y) {
115             return dfn[x] < dfn[y];
116         });
117
118         int ans = -2 * s;
119         for (int i = 1; i <= s; i++) {
120             int x = p[i], y = p[(i == s ? 1 : i + 1)];
121             int lca = qlca(x, y);
122             ans += dis[x] + dis[y] - 2 * dis[lca];
123         }
124
125         if (qlca(p[1], p[s]) <= n) {
126             ans += 2;
127         }
128         assert(ans % 2 == 0);
129         cout << ans / 2 << '\n';
130     }
131 }

```

IV. 利用强连通分量求解 2-sat 问题

2-SAT 问题，即要求解一组逻辑值 $x_1, x_2, \dots, x_n$ ，使得一个特定的每一项大小都为 2 的最小项逻辑表达式的值为真。

常用解决方法

Tarjan SCC 缩点

算法考究在建图这点，我们举个例子来讲：

假设有 a_1, a_2 和 b_1, b_2 两对，已知 a_1 和 b_2 间有矛盾，于是为了方案自治，由于两者中必须选一个，所以我们要拉两条有向边 (a_1, b_1) 和 (b_2, a_2) 表示选了 a_1 则必须选 b_1 ，选了 b_2 则必须选 a_2 才能够自治。

然后通过这样子建边我们跑一遍 Tarjan SCC 判断是否有一个集合中的两个元素在同一个 SCC 中，若有则输出不可能，否则输出方案。构造方案只需要把几个不矛盾的 SCC 拼起来就好了。

输出方案时可以通过变量在图中的拓扑序确定该变量的取值。如果变量 x 的拓扑序在 $\neg x$ 之后，那么取 x 值为真。应用到 Tarjan 算法的缩点，即 x 所在 SCC 编号在 $\neg x$ 之前时，取 x 为真。因为 Tarjan 算法求强连通分量时使用了栈，所以 Tarjan 求得的 SCC 编号相当于反拓扑序。

显然地，时间复杂度为 $O(n + m)$ 。

```
1 struct Two_Sat {
2     int n, cnt_id, cnt_clr, tp;
3     vector<vector<int>> G;
4     vector<int> dfn, low, clr, st, inst;
5     Two_Sat(int _n) {
6         n = _n;
7         cnt_id = tp = cnt_clr = 0;
8         G.resize(2 * n + 1);
9         dfn.resize(2 * n + 1);
10        low.resize(2 * n + 1);
11        clr.resize(2 * n + 1);
12        st.resize(2 * n + 1);
13        inst.resize(2 * n + 1);
14        for (int i = 1; i <= 2 * n; i++) {
15            G[i].clear();
16            dfn[i] = low[i] = clr[i] = st[i] = inst[i] = 0;
17        }
18    }
19    void addlim(int x, bool px, int y, bool py) { // (0/1)x v (0/1)y
20        G[x + px * n].push_back(y + (1 - py) * n);
21        G[y + py * n].push_back(x + (1 - px) * n);
```

```

22     }
23     void tarjan(int u) {
24         dfn[u] = low[u] = ++cnt_id;
25         st[++tp] = u; inst[u] = true;
26         for (auto v : G[u]) {
27             if (!dfn[v]) {
28                 tarjan(v);
29                 low[u] = min(low[u], low[v]);
30             } else if (inst[v]) {
31                 low[u] = min(low[u], dfn[v]);
32             }
33         }
34         if (dfn[u] == low[u]) {
35             cnt_clr++;
36             do {
37                 inst[st[tp]] = false;
38                 clr[st[tp]] = cnt_clr;
39             } while (st[tp--] != u);
40         }
41     }
42     vector<bool> Tres;
43     bool process() {
44         for (int i = 1; i <= 2 * n; i++) {
45             if (!dfn[i]) {
46                 tarjan(i);
47             }
48         }
49         for (int i = 1; i <= n; i++) {
50             if (clr[i] == clr[i + n]) {
51                 return false;
52             }
53         }
54
55         Tres.resize(n + 1, false);
56         for (int i = 1; i <= n; i++) {
57             if (clr[i] < clr[i + n]) {
58                 Tres[i] = true;
59             }
60         }
61         return true;
62     }
63 };

```

三. 图上计数相关

I. 无向图三元环计数

先给无向边定向，度数小的指向度数大的，度数相同编号小的指向编号大的，由于形成偏序关系，所以有向边不会成环。

枚举过程如代码中所示。

复杂度分析考虑根号分治，原图上度数小于等于 $\sqrt{m}$ 的结点最多只有 $\sqrt{m}$ 条出边，原图上度数大于 $\sqrt{m}$ 的结点最多也只有 $\sqrt{m}$ 条出边（不可能有超过 $\sqrt{m}$ 个度数比它还大的）。

所以总时间复杂度等于 $O(\sum_i out_i) = O(m\sqrt{m})$ 。

```
1  const int MAXN = 1e5 + 10;
2  const int MAXM = 2e5 + 10;
3
4  int n, m;
5  int from[MAXN], to[MAXN];
6  vector<int> G[MAXN];
7  int deg[MAXN];
8  bool vis[MAXN];
9
10 int main() {
11     cin >> n >> m;
12     for (int i = 1; i <= m; i++) {
13         cin >> from[i] >> to[i];
14         deg[from[i]]++; deg[to[i]]++;
15     }
16     for (int i = 1; i <= m; i++) {
17         int u = from[i], v = to[i];
18         if (deg[u] > deg[v] || (deg[u] == deg[v] && u > v)) {
19             swap(u, v);
20         }
21         G[u].push_back(v);
22     }
23
24     int ans = 0;
25     for (int u = 1; u <= n; u++) {
26         for (int v : G[u]) {
27             vis[v] = true;
28         }
29
30         for (int v : G[u])
31             for (int k : G[v])
32                 if (vis[k]) {
33                     ans++;
34                 }
```

```
34         }
35
36         for (int v : G[u]) {
37             vis[v] = false;
38         }
39     }
40     cout << ans << '\n';
41
42     return 0;
43 }
44
```

四. 网络流相关

I. Dinic 算法求最大流

dinic 算法一般图的最差时间复杂度为 $O(n^2m)$ ，单次增广最差时间复杂度为 $O(nm)$ 。但在类二分图上的时间复杂度仅为 $O(m\sqrt{n})$ 。

```
1 namespace Dinic {
2     const int MAXN = 2e2 + 10;
3     const int MAXM = 2e5 + 10;
4     const int INF = (1ll << 31) - 1;
5
6     struct Edge {
7         int to, w, nxt;
8     };
9
10    int n, m, S, T, cnt_e = 1;
11    int head[MAXN], curh[MAXN], dep[MAXN];
12    Edge e[MAXM];
13
14    void init(int _n) {
15        n = _n;
16        for (int i = 0; i <= n; i++) {
17            head[i] = 0;
18        }
19        cnt_e = 1;
20    }
21
22    void addedge(int u, int v, int w, bool flg) {
23        e[++cnt_e] = {v, w, head[u]};
24        head[u] = cnt_e;
25        if (flg) {
26            addedge(v, u, 0, false);
27        }
28    }
29
30    bool bfs() {
31        for (int i = 1; i <= n; i++) {
32            curh[i] = head[i];
33            dep[i] = -1;
34        }
35
36        queue<int> q;
37        dep[S] = 0; q.push(S);
38
39        while (!q.empty()) {
40            int u = q.front();
```

```

41         q.pop();
42
43         for(int i = head[u]; i; i = e[i].nxt) {
44             int v = e[i].to, w = e[i].w;
45             if(!w || dep[v] != -1) {
46                 continue;
47             }
48
49             dep[v] = dep[u] + 1;
50             q.push(v);
51
52             if (v == T) {
53                 return true;
54             }
55         }
56     }
57     return dep[T] != -1;
58 }
59
60 ll dfs(int u, int flw) {
61     if (u == T || !flw) {
62         return flw;
63     }
64     ll res = 0;
65     for (int &i = curh[u]; i; i = e[i].nxt) {
66         int v = e[i].to, w = e[i].w;
67         if(!w || dep[v] != dep[u] + 1) {
68             continue;
69         }
70         int k = dfs(v, min(w, flw));
71
72         e[i].w -= k;
73         e[i ^ 1].w += k;
74         flw -= k;
75         res += k;
76
77         if (!flw) {
78             break;
79         }
80     }
81     return res;
82 }
83
84 ll dinic(int s, int t) {
85     S = s; T = t;
86     ll res = 0;
87     while (bfs()) {
88         res += dfs(S, INF);

```

```
89         }
90         return res;
91     }
92 } using namespace Dinic;
```


II. SSP 算法求最小费用最大流

其实是伪多项式时间的算法，时间复杂度上界为 $O(nmf)$ ，但只要不是刻意对着这个算法卡就跑不满，实际效率还可以。

```
1 namespace MCMF {
2     const int MAXN = 5e3 + 10;
3     const int MAXM = 2e5 + 10;
4     const int INF = (1ll << 31) - 1;
5
6     struct Edge {
7         int to, w, c, nxt;
8     };
9
10    int n, m, S, T, cnt_e = 1; ll cost;
11    int head[MAXN], curh[MAXN], dis[MAXN];
12    bool inq[MAXN];
13    Edge e[MAXM];
14
15    void init(int _n) {
16        n = _n;
17        cnt_e = 1;
18        cost = 0;
19    }
20
21    void addedge(int u, int v, int w, int c, bool flg) {
22        e[++cnt_e] = {v, w, c, head[u]};
23        head[u] = cnt_e;
24        if (flg) {
25            addedge(v, u, 0, -c, false);
26        }
27    }
28
29    bool spfa() {
30        for (int i = 1; i <= n; i++) {
31            curh[i] = head[i];
32            dis[i] = INF;
33        }
34
35        queue<int> q;
36        dis[S] = 0; q.push(S); inq[S] = true;
37        while (!q.empty()) {
38            int u = q.front(); q.pop();
39            inq[u] = false;
40
41            for (int i = head[u]; i; i = e[i].nxt) {
42                int v = e[i].to, w = e[i].w, c = e[i].c;
```

```

43         if(!w || dis[u] + c >= dis[v]) {
44             continue;
45         }
46         dis[v] = dis[u] + c;
47         if (!inq[v]) {
48             q.push(v);
49             inq[v] = true;
50         }
51     }
52 }
53 return dis[T] != INF;
54 }
55
56 ll dfs(int u, int flw) {
57     if (u == T || !flw) {
58         return flw;
59     }
60
61     ll res = 0;
62     inq[u] = true;
63     for (int &i = curh[u]; i; i = e[i].nxt) {
64         int v = e[i].to, w = e[i].w, c = e[i].c;
65         if (!w || inq[v] || dis[u] + c != dis[v]) {
66             continue;
67         }
68         int k = dfs(v, min(w, flw));
69
70         cost += 1ll * c * k;
71         e[i].w -= k;
72         e[i ^ 1].w += k;
73         flw -= k;
74         res += k;
75
76         if (!flw) {
77             break;
78         }
79     }
80     inq[u] = false;
81     return res;
82 }
83
84 pll mcmf(int s, int t) {
85     S = s; T = t;
86     ll mxflw = 0;
87     while (spfa()) {
88         mxflw += dfs(S, INF);
89     }
90     return {mxflw, cost};

```

```
91     }  
92  
93 } using namespace MCMF;
```

五. 树上问题

I. 树上启发式合并 (Dsu on tree)

每个点到根的路径上不会有超过 $O(\log n)$ 条轻边，这保证了时间复杂度的正确性。

```
1  vector sz(n + 1, 0), dep(n + 1, 0), mxson(n + 1, 0);
2  auto dfs_init = [&](auto &&self, int u, int fa, int d) -> void {
3      sz[u] = 1;
4      dep[u] = d;
5      int mx = 0;
6      for (auto v : G[u]) {
7          if (v == fa) {
8              continue;
9          }
10         self(self, v, u, d + 1);
11         sz[u] += sz[v];
12         if (mx < sz[v]) {
13             mx = sz[v];
14             mxson[u] = v;
15         }
16     }
17 };
18 dfs_init(dfs_init, 1, 0, 1);
19
20 ll ans = 0;
21 BIT<int> trcnt; trcnt.init(n);
22 BIT<ll> trval; trval.init(n);
23 auto add = [&](auto &&self, int u, int fa, int op) -> void {    // 将子树中每个点的贡献添加到“桶”里面，先不算答案
24     trcnt.upd(dep[u], op);
25     trval.upd(dep[u], op * dep[u]);
26
27     for (auto v : G[u]) {
28         if (v == fa) {
29             continue;
30         }
31         self(self, v, u, op);
32     }
33 };
34 auto addans = [&](auto &&self, int u, int fa, int d) -> void {    // 计算子树中每个点与“桶”里面已有的贡献的答案
35     ll val0 = trval.query(1, dep[u]), cnt0 = trcnt.query(1, dep[u]);
36     ll cnt1 = trcnt.query(dep[u] + 1, n);
37     ans += 2 * (val0 - cnt0 * d) - cnt0;
38     ans += cnt1 * (2 * (dep[u] - d) - 1);
39     for (auto v : G[u]) {
```

```

40         if (v == fa) {
41             continue;
42         }
43         self(self, v, u, d);
44     }
45 };
46
47 auto dfs_calc = [&](auto &&self, int u, int fa, bool ismxson) -> void {
48     for (auto v : G[u]) {
49         if (v == fa || v == mxson[u]) {
50             continue;
51         }
52         self(self, v, u, false);
53     }
54     if (mxson[u]) {
55         self(self, mxson[u], u, true);
56     }
57     for (auto v : G[u]) {
58         if (v == fa || v == mxson[u]) {
59             continue;
60         }
61         addans(addans, v, u, dep[u]);
62         add(add, v, u, 1);
63     }
64
65     trcnt.upd(dep[u], 1);
66     trval.upd(dep[u], dep[u]);
67     if (!ismxson) {
68         add(add, u, fa, -1);
69     }
70 };
71 dfs_calc(dfs_calc, 1, 0, false);

```

II. 轻重链剖分

没啥好说的，直接上代码。

```
1  vector<int> G[MAXN];
2  int val1[MAXN], val2[MAXN];
3  int mxson[MAXN], fa[MAXN], dep[MAXN], sz[MAXN];
4  void dfs_init(int u, int fz, int d) {
5      int Mx = 0;
6      sz[u] = 1; dep[u] = d; fa[u] = fz;
7      for (auto v : G[u]) {
8          if (v == fz) {
9              continue;
10         }
11         dfs_init(v, u, d + 1);
12         sz[u] += sz[v];
13         if (sz[v] > Mx) {
14             Mx = sz[v];
15             mxson[u] = v;
16         }
17     }
18 }
19 int cnt_id;
20 int tp[MAXN], idx[MAXN];
21 void dfs_tp(int u, int tpfa) {
22     tp[u] = tpfa;
23     idx[u] = ++cnt_id;
24     val2[idx[u]] = val1[u];
25
26     if (mxson[u]) {
27         dfs_tp(mxson[u], tpfa);
28     }
29     for (auto v : G[u]) {
30         if (idx[v]) {
31             continue;
32         }
33         dfs_tp(v, v);
34     }
35 }
36
37 void Tree_add(int x, int y, int v) {
38     while (tp[x] != tp[y]) {
39         if (dep[tp[x]] < dep[tp[y]]) {
40             swap(x, y);
41         }
42         tr.upd(1, 1, n, idx[tp[x]], idx[x], v);
43         x = fa[tp[x]];
```

```

44     }
45     if (dep[x] > dep[y]) {
46         swap(x, y);
47     }
48     tr.upd(1, 1, n, idx[x], idx[y], v);
49 }
50
51 int Tree_query(int x, int y) {
52     ll res = 0;
53     while (tp[x] != tp[y]) {
54         if (dep[tp[x]] < dep[tp[y]]) {
55             swap(x, y);
56         }
57         res = (res + tr.query(1, 1, n, idx[tp[x]], idx[x])) % Mod;
58         x = fa[tp[x]];
59     }
60     if (dep[x] > dep[y]) {
61         swap(x, y);
62     }
63     res = (res + tr.query(1, 1, n, idx[x], idx[y])) % Mod;
64     return res;
65 }

```

III. 树哈希

超强防卡版。

```
1  const ull mask = mt19937_64(time(0))();
2  auto shift = [&](ull x) -> ull {
3      x ^= mask;
4      x ^= x << 13;
5      x ^= x >> 7;
6      x ^= x << 17;
7      x ^= mask;
8      return x;
9  };
10 vector sub(n + 1, 0llu), rt(n + 1, 0llu);
11 auto dfs_sub = [&](auto&& self, int u, int fa) -> void {
12     sub[u] = 1;
13     for (auto v : G[u]) {
14         if (v == fa) {
15             continue;
16         }
17         self(self, v, u);
18         sub[u] += shift(sub[v]);
19     }
20 };
21 auto dfs_rt = [&](auto&& self, int u, int fa) -> void {
22     for (auto v : G[u]) {
23         if (v == fa) {
24             continue;
25         }
26         rt[v] = sub[v] + shift(rt[u] - shift(sub[v]));
27         self(self, v, u);
28     }
29 };
30 auto dfs_hsh = [&](auto&& self, int u, int fa) -> ull {
31     ull res = shift(rt[u]);
32     for (auto v : G[u]) {
33         if (v == fa) {
34             continue;
35         }
36         res += shift(self(self, v, u));
37     }
38     return res;
39 };
```


IV. 点分治

关于点分树的一些性质，通过一个例题来讲解。

LuoguP6329 【模板】点分树 | 震波

- 给一棵 n 的点的树，有点权，有 q 次操作。每次修改一个点 x 的点权或查询到给定点 x 距离 $\leq k$ 的所有点的点权和。

有的问题我们不是非常关心树的形态特点，比如路径问题，联通块问题，寻找关键点问题等等，以路径问题为例，我们不一定非得查到 p, q 的 LCA 才可以处理 p, q 的路径信息，相反，我们可以随便从这个路径上寻找一个分割点 t ，只要我们可以快速的处理 p 到 t 和 q 到 t 的信息，我们就可以处理 p 到 q 的信息。

而点分树恰恰就是对原树做了这样的映射。

它有以下性质：

1. 它的高度与点分治的深度一样，只有 $\log n$ 级别，这个性质很关键，由于它的高度只有 $\log n$ ，所以我们可以搞出各种各样在一般树论里过不去的暴力做法，比如说对每个点开个包含子树中所有点的 vector，空间复杂度也只有。
2. 对于任意两点 u, v ，唯一可以确定的是 u, v 在点分树上的 LCA 一定在 $u \rightarrow v$ 的路径上。换句话说， $dis(u, v) = dis(u, lca) + dis(lca, v)$ 。

回到这题来，我们要求 $\sum_{dis(x,y) \leq k} a_y$ 。

考虑枚举 x, y 在点分树上的 LCA z （这显然是 $\log n$ 级别的），根据上面的推论有 $dis(x, y) = dis(x, z) + dis(y, z)$ 。

$$\text{故 } ans = \sum_{dis(x,z)+dis(z,y) \leq k \& LCA(x,y)=z} a_y = \sum_{dis(z,y) \leq k-dis(x,z) \& LCA(x,y)=z} a_y$$

考虑什么样的 y 满足 $LCA(x, y) = z$ ，显然符合要求 y 组成的集合就是 z 的子树抠掉 z 在 x 方向上的儿子 s 的子树。而我们要求这个点集中到 z 的距离 $\leq k - dis(x, z)$ 的点权和。显然可以拿 z 的子树内到 z 的距离 $\leq k - dis(x, z)$ 的点权和 - s 子树中到 z 的距离 $\leq k - dis(x, z)$ 的点权和。

对每个点 x 建一棵动态开点线段树，下标为 i 的位置维护 x 子树内所有 $dis(x, z) = i$ 的 a_z 的和。

那么求 z 子树内到 z 的距离 $\leq k - dis(x, z)$ 的点权和就在对应线段树上查个区间和就 ok 了。

那 z 在 x 方向上的儿子 s 的子树怎么办呢？

初学点分树的萌新（例如我）很容易进入一个误区，那就是这东西可以在 s 对应的线段树上查 $[0, k - dis(x, z) - 1]$ 的和。但这显然是错的，因为两点在点分树上的距离与两点在原树上的距离没有一丁点联系。到 s 距离 $\leq k - dis(x, z) - 1$ ，并不意味着到 z 距离 $\leq k - dis(x, z)$ 。

那么正解是什么呢？考虑对于每个点再建立一棵动态开点线段树，线段树上下标为 i 的位置维护 x 子树内到 f_{a_x} 距离 $= i$ 的点权和。解决 z 在 x 方向上的儿子 s 的子树的问题只需在点 s 的线段树上查询 $[0, k - dis(x, z)]$ 的和就行了。

代码前期部分主要是要找重心，然后分治，建立点分树，后面就是普通的暴力跳点分树上的 f_a 然后在权值线段树上进行修改。

直接放一个别人的代码。

```
1 #include <bits/stdc++.h>
2 using namespace std;
```

```

3 #define fi first
4 #define se second
5 #define fz(i,a,b) for(int i=a;i<=b;i++)
6 #define fd(i,a,b) for(int i=a;i>=b;i--)
7 #define ffe(it,v) for(__typeof(v.begin()) it=v.begin();it!=v.end();it++)
8 #define fill0(a) memset(a,0,sizeof(a))
9 #define fill1(a) memset(a,-1,sizeof(a))
10 #define fillbig(a) memset(a,63,sizeof(a))
11 #define pb push_back
12 #define ppb pop_back
13 #define mp make_pair
14 template<typename T1,typename T2> void chkmin(T1 &x,T2 y){if(x>y) x=y;}
15 template<typename T1,typename T2> void chkmax(T1 &x,T2 y){if(x<y) x=y;}
16 typedef pair<int,int> pii;
17 typedef long long ll;
18 template<typename T> void read(T &x){
19     x=0;char c=getchar();T neg=1;
20     while(!isdigit(c)){if(c=='-') neg=-1;c=getchar();}
21     while(isdigit(c)) x=x*10+c-'0',c=getchar();
22     x*=neg;
23 }
24 const int MAXN=1e5;
25 const int MAXP=5e6;
26 const int LOG_N=17;
27 const int INF=1e9;
28 int n,qu,a[MAXN+5];
29 int hd[MAXN+5],to[MAXN*2+5],nxt[MAXN*2+5],ec=0;
30 void adde(int u,int v){to[++ec]=v;nxt[ec]=hd[u];hd[u]=ec;}
31 int fa[MAXN+5][LOG_N+2],dep[MAXN+5];
32 void dfs0(int x,int f){        // 预处理 dep 等信息
33     fa[x][0]=f;
34     for(int e=hd[x];e=e=nxt[e]){
35         int y=to[e];if(y==f) continue;
36         dep[y]=dep[x]+1;dfs0(y,x);
37     }
38 }
39 int getlca(int x,int y){        // 求原树上的 lca, 用来算距离
40     if(dep[x]<dep[y]) swap(x,y);
41     for(int i=LOG_N;~i;i--) if(dep[x]-(1<<i)>=dep[y]) x=fa[x][i];
42     if(x==y) return x;
43     for(int i=LOG_N;~i;i--) if(fa[x][i]!=fa[y][i]) x=fa[x][i],y=fa[y][i];
44     return fa[x][0];
45 }
46 int getdis(int x,int y){return dep[x]+dep[y]-(dep[getlca(x,y)]<<1);}
47 int siz[MAXN+5],mx[MAXN+5],cent=0;
48 bool vis[MAXN+5];
49 void findcent(int x,int f,int tot){        // 找重心
50     siz[x]=1;mx[x]=0;

```

```

51     for(int e=hd[x];e=e=nxt[e]){
52         int y=to[e];if(y==f||vis[y]) continue;
53         findcent(y,x,tot);chkmax(mx[x],siz[y]);siz[x]+=siz[y];
54     } chkmax(mx[x],tot-siz[x]);
55     if(mx[x]<mx[cent]) cent=x;
56 }
57 int dfa[MAXN+5];
58 void divcent(int x,int tot){ // 点分治, 建立点分树
59 // printf("%d\n",x);
60     vis[x]=1;
61     for(int e=hd[x];e=e=nxt[e]){
62         int y=to[e];if(vis[y]) continue;
63         cent=0;int sz=(siz[y]<siz[x]?siz[x):(tot-siz[x]);
64         findcent(y,x,sz);dfa[cent]=x;divcent(cent,sz);
65     }
66 }
67 struct segtree{ // 权值线段树
68     int rt[MAXN+5],ncnt=0;
69     struct node{int ch[2],val;} s[MAXP+5];
70     void modify(int &k,int l,int r,int p,int x){
71         if(!k) k=++ncnt;
72         if(l==r){s[k].val+=x;return;}
73         int mid=(l+r)>>1;
74         if(p<=mid) modify(s[k].ch[0],l,mid,p,x);
75         else modify(s[k].ch[1],mid+1,r,p,x);
76         s[k].val=s[s[k].ch[0]].val+s[s[k].ch[1]].val;
77     }
78     int query(int k,int l,int r,int ql,int qr){
79         if(!k) return 0;
80         if(ql<=l&&r<=qr) return s[k].val;
81         int mid=(l+r)>>1;
82         if(qr<=mid) return query(s[k].ch[0],l,mid,ql,qr);
83         else if(ql>mid) return query(s[k].ch[1],mid+1,r,ql,qr);
84         else return
85         query(s[k].ch[0],l,mid,ql,mid)+query(s[k].ch[1],mid+1,r,mid+1,qr);
86     }
87 } w1,w2;
88 void modify(int x,int v){
89     int cur=x;
90     while(cur){
91         w1.modify(w1.rt[cur],0,n-1,getdis(cur,x),v);
92         if(dfa[cur]) w2.modify(w2.rt[cur],0,n-1,getdis(dfa[cur],x),v);
93         cur=dfa[cur];
94     }
95 }
96 int query(int x,int k){
97     int cur=x,pre=0,ret=0;
98     while(cur){

```

```

98         if(getdis(cur,x)>k){
99             pre=cur;cur=dfa[cur];continue;
100         }
101         ret+=w1.query(w1.rt[cur],0,n-1,0,k-getdis(cur,x));
102         if(pre) ret-=w2.query(w2.rt[pre],0,n-1,0,k-getdis(cur,x));
103         pre=cur;cur=dfa[cur];
104     } return ret;
105 }
106 int main(){
107     scanf("%d%d",&n,&qu);
108     for(int i=1;i<=n;i++) scanf("%d",&a[i]);
109     for(int i=1;i<=n;i++){int u,v;scanf("%d%d",&u,&v);adde(u,v);adde(v,u);}
110     dfs0(1,0);for(int i=1;i<=LOG_N;i++) for(int j=1;j<=n;j++)
111         fa[j][i]=fa[fa[j][i-1]][i-1];
112     mx[0]=INF;cent=0;findcent(1,0,n);divcent(cent,n);
113     // for(int i=1;i<=n;i++) printf("%d\n",dfa[i]);
114     for(int i=1;i<=n;i++) modify(i,a[i]);
115     int preans=0;
116     while(qu--){
117         int opt,x,y;scanf("%d%d%d",&opt,&x,&y);
118         x^=preans;y^=preans;
119         if(opt==0){preans=query(x,y);printf("%d\n",preans);}
120         else{modify(x,y-a[x]);a[x]=y;}
121     }
122     return 0;
123 }

```

六. 图论杂项

I. 欧拉路的判定与求解

欧拉路存在的充要条件：

1. 图是连通的，若不连通不可能一次性遍历所有边。
2. 对于无向图：有且仅有两个点，与其相连的边数为奇数，其他点相连边数皆为偶数；或所有点皆为偶数边点。

对于两个奇数点，一个为起点，一个为终点。起点需要出去，终点需要进入，故其必然与奇数个边相连。

如果存在这样一个欧拉路，其所有的点相连边数都为偶数，那说明它是欧拉回路。

因为此时它的起点即是终点，出去后还会回来，刚好形成偶数边。

3. 对于有向图：除去起点和终点，所有点的出度与入度相等。起点出度比入度大 1，终点入度比出度大 1。若起点终点出入度也相同，则为欧拉回路。

Hierholzer 算法求欧拉回路。

```
1  vector<int> ans;
2  void Hierholzer(int u) {
3      while (!G[u].empty()) {
4          auto v = *G[u].begin();
5          G[u].erase(G[u].begin());
6          G[v].erase(G[v].find(u));
7          Hierholzer(v);
8      }
9      ans.push_back(u);
10 }
```

II. 平面图的判定

平面图判定定理：

- 如果一个图 G 不存在任意一个子图为五阶完全图或者左右两部都为大小都为 3 的完全二分图，那么图 G 就是平面图。

III. 最小斯坦纳树

给定若干个结点构成的集合 P ，要求从原图 $G = (V, E)$ 中找出一个包含 P 中所有点的连通子图，且边权和最小。

考虑使用状压 dp，设 $f_{i,S}$ 代表以 i 为根，包含集合 S 中所有点的最小代价。

一棵以 i 为根的树有两种情况，第一种是 i 的度数为 1，可以考虑枚举树上与 i 相邻的点 j ，则

$$f_{i,S} \leftarrow f_{j,S} + w_{i,j}$$

对于度数大于 1 的情况，可以划分成若干个子树，则

$$f_{i,S} \leftarrow f_{i,T} + f_{i,S-T}$$

对于第一种转移，发现松弛过程与最短路一致，可以跑一个全源的 dij。对于第二种转移枚举子集即可。

总时间复杂度 $O(n \times 3^{|P|} + m \log m \times 2^{|P|})$ 。

```
1  memset(f, 0x3f, sizeof(f));
2  for (int i = 1; i <= k; i++) {
3      cin >> p[i];
4      f[1 << (i - 1)][p[i]] = 0;
5  }
6
7  for (int s = 1; s < (1 << k); s++) {
8      for (int i = 1; i <= n; i++) {
9          for (int t = (s - 1) & s; t; t = (t - 1) & s) {
10             chk_min(f[s][i], f[t][i] + f[s ^ t][i]);
11         }
12         pq.push({-f[s][i], i});
13     }
14     dij(s);
15 }
16
17 int ans = 1e9;
18 for (int i = 1; i <= n; i++) {
19     chk_min(ans, f[(1 << k) - 1][i]);
20 }
21 cout << ans << '\n';
```

数学部分

一. 数论基础

I. 扩展欧几里得算法 (exgcd)

求方程组 $ax + by = \gcd(a, b)$ 的一组解 (x_0, y_0) 。

```
1  int exgcd(int a, int b, int &x, int &y) {
2      if (!b) {
3          x = 1;
4          y = 0;
5          return a;
6      }
7      int g = exgcd(b, a % b, x, y);
8      int tmp = x; x = y;
9      y = tmp - a / b * y;
10     return g;
11 }
```


II. 扩展中国剩余定理 (exCRT)

扩展中国剩余定理推导如下，解决 n 组同余方程的方程组：

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \dots \\ x \equiv r_n \pmod{m_n} \end{cases}$$

这里的 m_i 并不一定保证两两互质，是没有任何限制的。

扩展中国剩余定理的本质思想其实就是把两个同余方程合并成为一个同余方程，考虑：

$$\begin{cases} a \equiv r_1 \pmod{m_1} \\ a \equiv r_2 \pmod{m_2} \end{cases}$$

假设这两个方程存在一个等价的方程 $a \equiv r \pmod{m}$ 。

那么设 $a = k_1 m_1 + r_1 = k_2 m_2 + r_2$ ，我们得到： $k_1 m_1 - k_2 m_2 = r_2 - r_1$ 。

现在我们要要求一组解 (k'_1, k'_2) 。

根据裴蜀定理，当 $\gcd(m_1, m_2) \nmid (r_2 - r_1)$ 时，该方程无整数解。

这样，在判断有解后，很容易通过扩展欧几里得算法求出一组解 (k'_1, k'_2) 。

这样，我们就求出了一个解 $a_0 = k'_1 m_1 + r_1$ 。

我们又发现， $a_0 + \text{lcm}(m_1, m_2)$ 也是一组合法解。

所以 $a \equiv a_0 \pmod{\text{lcm}(m_1, m_2)}$ ，但是我们需要证明这涵盖了所有合法解。

假设在模 $\text{lcm}(m_1, m_2)$ 的剩余系中存在 x, y 使得：

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \end{cases} \begin{cases} y \equiv r_1 \pmod{m_1} \\ y \equiv r_2 \pmod{m_2} \end{cases}$$

并且 $x \neq y$ 。那么，一定满足：

$$\begin{cases} x - y \equiv 0 \pmod{m_1} \\ x - y \equiv 0 \pmod{m_2} \end{cases}$$

所以 $\text{lcm}(m_1, m_2) \mid (x - y)$ ，又因为 x, y 均为模 $\text{lcm}(m_1, m_2)$ 的剩余系中的元素，所以只能存在 $x = y$ 一种情况。

所以在模 $\text{lcm}(m_1, m_2)$ 的剩余系中只能存在一个解。

所以 $a \equiv a_0 \pmod{\text{lcm}(m_1, m_2)}$ 是原同余方程组的通解。

至此，我们 $O(\log n)$ 实现了两个同余方程的合并，然后我们依次合并 n 个同余方程，就是扩展中国剩余定理。

```
1  const int MAXN = 1e5 + 10;
2
3  ll mul(ll x, ll y, ll p) {
4      int flg = 1;
5      if (y < 0) y = -y, flg = -1;
6      ll res = 0;
7      while (y) {
8          if (y & 1) res = (res + x) % p;
9          x = (x + x) % p;
10         y >>= 1;
11     }
12     return res * flg;
13 }
14
15 ll gcd(ll a, ll b) {
16     return !b ? a : gcd(b, a % b);
17 }
18
19 ll lcm(ll a, ll b) {
20     return a / gcd(a, b) * b;
21 }
22
23 ll exgcd(ll a, ll b, ll &x, ll &y) {
24     if (!b) return x = 1, y = 0, a;
25     ll g = exgcd(b, a % b, x, y);
26     ll tmp = x;
27     x = y, y = tmp - a / b * y;
28     return g;
29 }
30
31 int n;
32 ll a[MAXN], b[MAXN];
33
34 void excrt()
35 {
36     ll r1 = b[1], m1 = a[1];
37     for(int i = 2; i <= n; i++)
38     {
39         ll r2 = b[i], m2 = a[i];
40
41         ll k1, k2;
42         ll g = exgcd(m1, -m2, k1, k2);
43
44         m1 = lcm(m1, m2);
```

```

45
46     k1 = mul(k1, ((r2 - r1) / g), m1);
47     k2 = mul(k2, ((r2 - r1) / g), m1);
48
49     r1 = ((mul(k2, m2, m1) + r2) % m1 + m1) % m1;
50 }
51 printf("%lld\n", r1);
52 }
53
54 int main() {
55     scanf("%d", &n);
56     for(int i = 1; i <= n; i++)
57         scanf("%lld%lld", &a[i], &b[i]);
58     excrt();
59     return 0;
60 }

```

III. 欧拉定理与扩展欧拉定理

考虑使用群论的基本知识证明欧拉定理：若 $a \perp n$ ，则有

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

设 n 是一个正整数，令 $(G, \cdot)$ 为整数集合 $\mathbb{Z}$ 模 n 且与 n 互质的剩余类关于乘法构成的群，显然 $|G| = \varphi(n)$ 。

先简要证明一下 $(G, \cdot)$ 满足群的定义：

- 封闭性

若 a, b 均与 n 互质，根据唯一分解定理，有 $a \cdot b$ 与 n 互质，运算满足封闭性。

- 结合律

若 a, b, c 均与 n 互质，根据同余的理论，有 $(a \cdot b) \cdot c \equiv a \cdot (b \cdot c) \pmod{n}$ ，运算满足结合律。

- 存在幺元

显然 1 与 n 互质，所以 $1 \in G$ ，且满足 $1 \cdot a \equiv a \cdot 1 \equiv a \pmod{n}$ ，故 1 为群 G 的幺元；

- 存在逆元

若 a 与 n 互质，根据裴蜀定理可以推导出，存在正整数 $b \perp n$ 使得 $ab \equiv 1 \pmod{n}$ ，所以 G 中的所有元素均存在逆元。

由拉格朗日定理的推论

有限群 G 的任一元素 a 的阶 $o(a)$ 整除 G 的阶；于是 $a^{|G|} = e$ 。

我们立即得到

$$a^{|G|} \equiv 1 \pmod{n}$$

代入 $|G| = \varphi(n)$ 即可得到欧拉定理。

扩展欧拉定理描述如下（不再进行证明），这里只要求 a, m 为整数，不需要 a, m 互质。

$$a^b \equiv \begin{cases} a^b & , b \leq \varphi(m) \\ a^{b \bmod \varphi(m) + \varphi(m)} & , b > \varphi(m) \end{cases} \pmod{m}$$

IV. 阶与原根

定义

1. 阶：

对于 $a \in \mathbb{Z}, m \in \mathbb{N}^*, \gcd(a, m) = 1$ 。

满足同余式 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 称作 a 模 m 的阶，记作 $\delta_m(a)$ 。

2. 原根：

对于 $a \in \mathbb{Z}, m \in \mathbb{N}^*, \gcd(a, m) = 1$ 。

由欧拉定理，我们知道 $a^{\varphi(m)} \equiv 1 \pmod{m}$ 。若 $\delta_m(a) = \varphi(m)$ ，则称 a 为模 m 的原根。

性质

1. 阶：

「阶」性质 1： $a, a^2, a^3, \dots, a^{\delta_m(a)}$ 模 m 两两不同余。

「阶」性质 2： 若 $a^n \equiv 1 \pmod{m}$ ，则 $\delta_m(a) \mid n$ 。

推论： $a^p \equiv a^q \pmod{m}$ ，则 $p \equiv q \pmod{\delta_m(a)}$ 。

「阶」性质 3： 若 $a, b \in \mathbb{Z}, m \in \mathbb{N}^*, \gcd(a, m) = \gcd(b, m) = 1$ ，则 $\delta_m(ab) = \delta_m(a)\delta_m(b) \iff \gcd(\delta_m(a), \delta_m(b)) = 1$ 。

「阶」性质 4： 若 $a \in \mathbb{Z}, k \in \mathbb{N}, m \in \mathbb{N}^*, \gcd(a, m) = 1$ ，则 $\delta_m(a^k) = \frac{\delta_m(a)}{\gcd(\delta_m(a), k)}$ 。

2. 原根：

「原根」性质 1：（原根个数） 若一个数 m 有原根，则其原根个数为 $\varphi(\varphi(m))$ 。

「原根」性质 2：（原根存在定理） 一个数 m 存在原根当且仅当 $m \in \{2, 4, p^\alpha, 2p^\alpha \mid p \text{ is an odd prime}, \alpha \in \mathbb{N}^*\}$ 。

「原根」性质 3：（最小原根） 若一个数 m 有原根，则其最小原根一定是不多于 $m^{0.25}$ 级别的。
（这个性质是由我国数学家王元在 1959 年证明的）

证明

这里简单证明一下「原根」性质 1，因为在证明过程中的结论需要用于快速求一个数的所有原根。至于其他的证明，有空再回来补一补吧。

证明：「原根」性质 1

设 m 存在一个原根 g 。

首先，根据「阶」性质 1 可得：

$g, g^2, \dots, g^{\varphi(m)}$ 两两不同, 且都与 m 互质。因为能成为 m 原根的数至少与 m 互质, 这样 g 的次幂就覆盖了所有可能成为原根的数, 所以 $g^x, x \in [1, \varphi(m)]$ 一定能表示出 m 的所有原根。

接着, 根据「阶」性质 4 可得:

$$\delta_m(g^k) = \frac{\delta_m(g)}{\gcd(\delta_m(g), k)} = \frac{\varphi(m)}{\gcd(\varphi(m), k)}$$

如果 $\gcd(\varphi(m), k) = 1$, 则 $\delta_m(g^k) = \varphi(m)$, g^k 也是 m 的原根。

容易发现, 满足 $1 \leq k < \varphi(m)$ 且 $\gcd(\varphi(m), k) = 1$ 的 k 有 $\varphi(\varphi(m))$ 个。

Q.E.D.

求原根的算法流程:

下面是求一个数 m 的原根的算法流程:

1. 预处理:

1. 线性筛出 m 以内所有素数和所有数的欧拉函数。
2. 对于每一个 m 以内的素数, 求出所有不大于 m 的奇素数 p^α 和 $2p^\alpha$ 。
3. 判断 m 是否有原根。

2. 求最小原根 a :

1. 求出 $\varphi(m)$ 的所有因数。
2. 从小到大枚举与 m 互质的数 a 。
3. 对于 $\varphi(m)$ 的每个因数 x , 若 $a^x \equiv 1 \pmod{m}$ 且 $j \neq \varphi(m)$, 则 a 不是 m 的原根。

3. 求所有原根:

1. 枚举 $\varphi(m)$ 以内的所有数 i 。
2. 若 $\gcd(\varphi(m), i) = 1$, 则 a^i 为一个新的原根。

```

1 // 代码有些古老, 但是这里重在理论, 所以代码就没改
2
3 const int MAXM=1e6+10;
4
5 inline int gcd(int x,int y){ return !y ? x : gcd(y,x%y); }
6 inline ll ksm(ll x,ll y,ll p)
7 {
8     ll res=1;
9     while(y)
10    {
11        if(y&1) res=res*x %p;
12        x=x*x %p;
13        y>>=1;

```

```

14     }
15     return res;
16 }
17
18 int pcnt;
19 bool isnp[MAXM];
20 int p[MAXM];
21 int phi[MAXM];
22
23 inline void init(int n)
24 {
25     isnp[0]=isnp[1]=true;
26     phi[1]=1;
27     for(int i=2;i<=n;i++)
28     {
29         if(!isnp[i])
30         {
31             p[++pcnt]=i;
32             phi[i]=i-1;
33             for(int cur=i;(ll)cur*i<=n;cur*=i)
34                 phi[cur*i]=i*phi[cur];
35         }
36         for(int j=1;j<=pcnt && (ll)i*p[j]<=n;j++)
37         {
38             isnp[i*p[j]]=true;
39             int x=i,y=p[j];
40             while(x%p[j]==0) x/=p[j],y*=p[j];
41             phi[x*y]=phi[x]*phi[y];
42             if(i%p[j]==0) break;
43         }
44     }
45 }
46
47 inline void solve()
48 {
49     int m,d;
50     scanf("%d%d",&m,&d);
51     bool fg=false;
52     for(int i=1;i<=pcnt && p[i]<=m;i++)
53     {
54         if(p[i]%2==0) continue;
55         ll cur=p[i];
56         for(;cur<=m;cur*=p[i])
57             if(cur==m || 2*cur==m)
58             {
59                 fg=true;
60                 break;
61             }

```

```

62     }
63     if(m!=2 && m!=4 && !fg)
64     {
65         puts("0\n");
66         return ;
67     }
68
69     vector<int> yueshu;
70     for(int i=1;i*i<=phi[m];i++)
71     {
72         if(phi[m]%i) continue;
73         yueshu.push_back(i);
74         if(i*i!=phi[m]) yueshu.push_back(phi[m]/i);
75     }
76     sort(yueshu.begin(),yueshu.end());
77
78     int a=-1;
79     for(int i=1;i<=m;i++)
80     {
81         if(gcd(i,m)>1) continue;
82         fg=true;
83         for(int j=0;j<(int)yueshu.size();j++)
84             if(ksm(i,yueshu[j],m)==1 && yueshu[j]!=phi[m])
85             {
86                 fg=false;
87                 break;
88             }
89         if(fg)
90         {
91             a=i;
92             break;
93         }
94     }
95
96     vector<int> pr;
97     for(int i=1;i<=phi[m];i++)
98     {
99         if(gcd(i,phi[m])>1) continue;
100         pr.push_back(ksm(a,i,m));
101     }
102     sort(pr.begin(),pr.end());
103
104     printf("%d\n",(int)pr.size());
105     for(int i=0;i<(int)pr.size();i++)
106         if((i+1)%d==0)
107             printf("%d ",pr[i]),fg=true;
108     puts("");
109 }

```



```
110
111 int main()
112 {
113     init(MAXM-10);
114     int T;
115     scanf("%d",&T);
116     while(T--) solve();
117     return 0;
118 }
```

V. 离散对数

BSGS 与 exBSGS

大步小步算法，利用了根号分治的思想，求解 $a^x \equiv b \pmod{p}$ 的最小整数解。

```
1 namespace BSGS {
2     const int INF = (1ll << 31) - 1;
3     int bsgs(int a, int p, int b, int coef) { // gcd(a, p) = 1, 找到 a ^
x \equiv b (mod p) 的最小整数解
4         a %= p;
5         b %= p;
6
7         int t = ceil(sqrt(p));
8         unordered_map<int, int> mp;
9         int cur = b;
10        for (int i = 0; i <= t; i++) {
11            mp[cur] = max(mp[cur], i + 1);
12            cur = 1ll * cur * a % p;
13        }
14
15        int mul = 1;
16        for (int i = 1; i <= t; i++) {
17            mul = 1ll * mul * a % p;
18        }
19        cur = mul;
20        int res = INF;
21        for (int i = 1; i <= t; i++) {
22            int v = 1ll * coef * cur % p;
23            if (mp[v]) {
24                res = min(res, i * t - (mp[v] - 1));
25            }
26            cur = 1ll * cur * mul % p;
27        }
28
29        if (res == INF) {
30            return -1;
31        }
32        return res;
33    }
34    int exbsgs(int a, int p, int b) { // gcd(a, p) != 1 时
35        a %= p;
36        b %= p;
37        if (b == 1 || p == 1) {
38            return 0;
39        }
40        int k = 0, coef = 1;
```

```

41     while (true) {
42         int g = __gcd(a, p);
43         if (g == 1) {
44             break;
45         }
46         if (b % g != 0) {
47             return -1;
48         }
49         k++; b /= g; p /= g;
50         coef = (1ll * coef * a / g) % p;
51         if (coef == b) {
52             return k;
53         }
54     }
55     int res = bsgs(a, p, b, coef);
56     if (res == -1) {
57         return -1;
58     } else {
59         return res + k;
60     }
61 }
62 }

```

二. 数论进阶——素数、数论函数与筛法

I. 线性筛

线性筛（筛 μ 函数）模板

```
1  const int N = 5e7;
2  const int MAXN = N + 10;
3
4  bool isnp[MAXN];
5  int pcnt, p[MAXN], mu[MAXN];
6  void init_eular(int n) {
7      isnp[1] = true; mu[1] = 1;
8      for (int i = 2; i <= n; i++) {
9          if (!isnp[i]) {
10             p[++pcnt] = i;
11             mu[i] = -1;
12         }
13         for (int j = 1; j <= pcnt && i * p[j] <= n; j++) {
14             int x = i, y = p[j];
15             while (x % p[j] == 0) {
16                 x /= p[j];
17                 y *= p[j];
18             }
19             if (x > 1) {
20                 mu[x * y] = mu[x] * mu[y];
21             }
22             isnp[x * y] = true;
23             if (i % p[j] == 0) {
24                 break;
25             }
26         }
27     }
28 }
```

II. 狄利克雷卷积与莫比乌斯反演的一些结论

反演过程

设 $g = f * 1$ ，那么 $f = f * 1 * \mu = g * \mu$ 。（其实这就是莫比乌斯反演的公式）

同理，设 $g = f * \text{id}^k$ ，设 $t(n) = \mu(n) \cdot n^k$ ，易证 $\text{id}^k * t = \epsilon$ ，所以 $f = f * \text{id}^k * t = g * t$ 。

常用数论函数

- 单位函数: $\epsilon(n) = [n = 1]$ (完全积性)
- 恒等函数: $\text{id}_k(n) = n^k$, $\text{id}_1(n)$ 通常简记作 $\text{id}(n)$ 。(完全积性)
- 常数函数: $1(n) = 1$ (完全积性)
- 除数函数: $\sigma_k(n) = \sum_{d|n} d^k \sigma_0(n)$ 通常简记作 $d(n)$ 或 $\tau(n)$, $\sigma_1(n)$ 通常简记作 $\sigma(n)$ 。
- 欧拉函数: $\varphi(n) = \sum_{i=1}^n [\gcd(i, n) = 1]$
- 莫比乌斯函数: $\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & \exists d > 1 : d^2 | n, \text{ 其中 } \omega(n) \text{ 表示 } n \text{ 的本质不同质因子个数} \\ (-1)^{\omega(n)} & \text{otherwise} \end{cases}$ 它是一个加性函数。

加性函数

此处加性函数指数论上的加性函数 (Additive function)。对于加性函数 f ，当整数 a, b 互质时，均有 $f(ab) = f(a) + f(b)$ 。应与代数中的加性函数 (Additive map) 区分。

洛谷

常用结论

$$\begin{aligned}\epsilon &= \mu * 1 \iff \epsilon(n) = \sum_{d|n} \mu(d) \\ d &= 1 * 1 \iff d(n) = \sum_{d|n} 1 \\ \sigma &= \text{id} * 1 \iff \sigma(n) = \sum_{d|n} d \\ \varphi &= \mu * \text{id} \iff \varphi(n) = \sum_{d|n} d \cdot \mu\left(\frac{n}{d}\right)\end{aligned}$$

洛谷

- 证明一下最后一个：

$$\begin{aligned}
 \varphi(n) &= \sum_{i=1}^n [\gcd(i, n) = 1] \\
 &= \sum_{i=1}^n \sum_{d \mid \gcd(i, n)} \mu(d) \\
 &= \sum_{d \mid n} \mu(d) \sum_{d \mid i, i \leq n} 1 \\
 &= \sum_{d \mid n} \mu(d) \frac{n}{d}
 \end{aligned}$$

- 下面是做题中遇到的一些重要结论：

1. 与 n 互质的数的和等于 $\frac{\varphi(n) * n}{2}$ ， $n = 1$ 时除外。

略证：考虑与 n 互质的数一定成对出现，即若 $x \perp n$ ，则 $(n - x) \perp n$ ，然后显然可证。

2. $\mathbf{d}(nm) = \sum_{x \mid n} \sum_{y \mid m} [\gcd(x, y) = 1]$ 。

略证：懒得写了，点这儿。

3. $\sum_{i=1}^n \lfloor \frac{n}{i} \rfloor = \sum_{i=1}^n \mathbf{d}(n)$ 。

4. $\sum_{i=1}^n \mu^2(n) = \sum_{d=1}^{\sqrt{n}} \mu(d) \lfloor \frac{n}{d^2} \rfloor$ 。

III. 杜教筛

解决问题

给定数论函数 $\mathbf{f}$ ，在低于线性的时间复杂度下求出 $\mathbf{S}(n) = \sum_{i=1}^n \mathbf{f}(i)$ 。

解决方法

假设我们有数论函数 $\mathbf{g}$ ，能快速对 $\mathbf{f} * \mathbf{g}$ 求前缀和，即能够快速求出 $\sum_{i=1}^n (\mathbf{f} * \mathbf{g})(i)$ 。

考虑 $\sum_{i=1}^n (\mathbf{f} * \mathbf{g})(i) = \sum_{i=1}^n \sum_{ab=i} \mathbf{f}(a)\mathbf{g}(b)$ 。

考虑交换求和顺序，改为先枚举 b ，可得原式即为

$$\sum_{b=1}^n \sum_{ab \leq n} \mathbf{f}(a)\mathbf{g}(b) = \sum_{b=1}^n \mathbf{g}(b) \sum_{a=1}^{\lfloor \frac{n}{b} \rfloor} \mathbf{f}(a) = \sum_{b=1}^n \mathbf{g}(b) \mathbf{S}(\lfloor \frac{n}{b} \rfloor)。$$

所以我们得到：

$$\begin{aligned} \sum_{i=1}^n (\mathbf{f} * \mathbf{g})(i) &= \sum_{b=1}^n \mathbf{g}(b) \mathbf{S}(\lfloor \frac{n}{b} \rfloor) \\ \sum_{i=1}^n (\mathbf{f} * \mathbf{g})(i) &= \mathbf{g}(1) \mathbf{S}(n) + \sum_{b=2}^n \mathbf{g}(b) \mathbf{S}(\lfloor \frac{n}{b} \rfloor) \\ \mathbf{g}(1) \mathbf{S}(n) &= \sum_{i=1}^n (\mathbf{f} * \mathbf{g})(i) - \sum_{b=2}^n \mathbf{g}(b) \mathbf{S}(\lfloor \frac{n}{b} \rfloor) \end{aligned}$$

所以我们只要能快速求出 $\mathbf{g}$ 和 $\mathbf{f} * \mathbf{g}$ 的前缀和，就可以依据这个柿子递归地求出 $\mathbf{S}(n)$ 了。

时间复杂度

数论分块的时候我们知道，对于 n 以内不同的 i ， $\lfloor \frac{n}{i} \rfloor$ 的取值最多有 $\sqrt{n}$ 种，又因为

$$\left\lfloor \frac{\lfloor \frac{n}{i} \rfloor}{j} \right\rfloor = \left\lfloor \frac{n}{ij} \right\rfloor, \text{ 所以:}$$

对于每次递归时计算的 $\mathbf{S}(n)$ ，需要 $O(\sqrt{n})$ 的复杂度，所以我们可以列出一下时间复杂度的柿子：

$$O\left(\sum_{i=1}^{\sqrt{n}} \sqrt{i} + \sum_{i=1}^{\sqrt{n}} \sqrt{\frac{n}{i}}\right)$$

这个柿子可以使用积分求出其近似值：

$$O\left(\int_1^{\sqrt{n}} \sqrt{x} \cdot dx + \int_1^{\sqrt{n}} \sqrt{\frac{n}{x}} \cdot dx\right) = O\left(\frac{2}{3}\sqrt{n}^{\frac{3}{2}} + \sqrt{n} \cdot 2\sqrt{n}^{\frac{1}{2}}\right) = O(n^{\frac{3}{4}})$$

然而这样做其实还不够优秀，假设我们提前预处理前 n^c 处的 $\mathbf{f}$ 的值，我们能得到以下的复杂度：

$$O\left(n^c + \int_1^{n^{1-c}} \sqrt{\frac{n}{x}} \cdot dx\right) = O\left(n^c + \sqrt{n} \cdot 2n^{\frac{1}{2}(1-c)}\right) = O(n^c + n^{1-\frac{1}{2}c})$$

此时，我们令 $c = 1 - \frac{1}{2}c$ ，解得 $c = \frac{2}{3}$ ，这样我们得到了杜教筛的时间复杂度下界： $O(n^{\frac{2}{3}})$ 。

```

1  int getS(int x) {
2      int res = calc(x); // \sum_{i=1}^x (f*g)(i)
3      for (int i = 2, r; i <= x; i = r + 1) {
4          r = x / (x / i);
5          res -= sumg(i, r) * getS(x / i);
6          i = r + 1;
7      }
8      res /= g(1);
9      return res;
10 }
```


三. 线性代数

I. 线性基

本质上是对二进制的每一位做模 2 意义下的高斯消元，消元之后最多只有 $\log V$ 行，矩阵中的所有非零行向量之间线性无关，所以线性基中的所有非零行向量构成了一组基底。

```
1 struct Line_Basis {
2     ll s[64];
3     int tot, cntdf;
4     Line_Basis() {
5         tot = cntdf = 0;
6         memset(s, 0, sizeof(s));
7     }
8     bool insert(ll x) {
9         for (int i = 63; i >= 0; i--) {
10             if ((x >> i) & 1) {
11                 if (!s[i]) {
12                     s[i] = x;
13                     tot++;
14                     return true;
15                 }
16                 else {
17                     x ^= s[i];
18                 }
19             }
20         }
21         cntdf++;
22         return false;
23     }
24     ll qmax() {
25         ll res = 0;
26         for (int i = 63; i >= 0; i--) {
27             if ((res >> i) & 1) {
28                 continue;
29             } else {
30                 res ^= s[i];
31             }
32         }
33         return res;
34     }
35     void init_qkth() {
36         for (int i = 0; i <= 63; i++) {
37             for (int j = 0; j < i; j++) {
38                 if ((s[i] >> j) & 1) {
39                     s[i] ^= s[j];
40                 }
41             }
42         }
43     }
44 }
```

```

41         }
42     }
43 }
44 ll qkth(ll k) {    // 查询线性基中所有基底的异或第 k 小
45     if (cntdf) {
46         if (k == 1) {
47             return 0;
48         }
49         k--;
50     }
51     if (k > (1ll << tot) - 1) {
52         return -1;
53     }
54
55     ll res = 0;
56     for (int i = 0; i <= 63; i++) {
57         if (s[i]) {
58             if (k & 1) {
59                 res ^= s[i];
60             }
61             k >>= 1;
62         }
63     }
64     return res;
65 }
66 } LB;

```

四. 数值算法

I. 拉格朗日插值

给定 $n + 1$ 个横坐标不同的点，求一个 n 次多项式在其他位置的取值，直接高斯消元法算多项式系数的时间复杂度是 $O(n^3)$ 的，有没有更快的方法呢。

有，就是拉格朗日插值法。

前置知识

首先由一个显然的结论，若 a 是整数，则有

$$x \equiv a \pmod{x - a}$$

根据同余的性质，若 k 是整数，则有

$$x^k \equiv a^k \pmod{x - a}$$

于是可以进一步得出结论，若 $f(x)$ 是关于 x 的多项式，则有

$$f(x) \equiv f(a) \pmod{x - a}$$

结论推导

设给定的 $n + 1$ 个点的横坐标分别为 $x_0, x_1, \dots, x_n$ ，根据上述结论，则有同余方程组

$$\begin{cases} f(x) \equiv f(x_0) \pmod{x - x_0} \\ f(x) \equiv f(x_1) \pmod{x - x_1} \\ \dots \\ f(x) \equiv f(x_n) \pmod{x - x_n} \end{cases}$$

要得到 $f(x)$ 的表达式，实际上就是解这个同余方程组，这就很自然地让我们联想到了中国剩余定理。

设 $N = \prod_{i=0}^n (x - x_i)$ ，则对于第 i 个同余式的基向量就是

$$e_i = \frac{N}{x - x_i} \left(\frac{N}{x - x_i} \right)^{-1}_{\text{mod } x - x_i} = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

所以用中国剩余定理合并起来就是

$$f(x) = \sum_{i=0}^n f(x_i) e_i = \sum_{i=0}^n f(x_i) \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

这样，我们只需要 $O(n^2)$ 的时间复杂度便可以求出 $f(x)$ 在其他位置的取值了，有一些题还有一些性质，可以做到 $O(n \log n)$ 甚至 $O(n)$ 。

一个例题

求 $S_k(n) = \sum_{i=0}^n i^k$, $1 \leq n \leq 10^9, 1 \leq k \leq 10^6$ 。

根据初中数学知识, 我们知道 $S_1(n) = \frac{n(n+1)}{2}$, $S_2(n) = \frac{n(n+1)(2n+1)}{6}$, $S_3(n) = \frac{n^2(n+1)^2}{4}$, 这些分别是关于 n 的 2, 3, 4 次多项式。

于是我们猜想 $S_k(n)$ 一定能表示成关于 n 的 $k+1$ 次多项式。

考虑第二类斯特林数, 对通常幂进行转化

$$i^k = \sum_{j=0}^k \binom{i}{j} \left\{ \begin{matrix} k \\ j \end{matrix} \right\} j! = \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} i^{\underline{j}}$$

带入 $S_k(n)$ 中可得

$$\begin{aligned} & \sum_{i=0}^n i^k \\ &= \sum_{i=0}^n \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} i^{\underline{j}} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \sum_{i=j}^n i^{\underline{j}} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} j! \sum_{i=j}^n \binom{i}{j} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} j! \sum_{i=0}^{n-j} \binom{i+j}{j} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} j! \binom{n+1}{j+1} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \frac{(n+1)^{\underline{j+1}}}{j+1} \end{aligned}$$

由此, 我们证明了 $S_k(n)$ 是关于 n 的 $k+1$ 次多项式。

所以我们只要求出 $S_k(1), S_k(2), \dots, S_k(k+2)$ 这 $k+2$ 个点的点值, 然后拉格朗日插值一下, 就可以求出 $S_k(n)$ 了。

那么拉格朗日插值的式子就是

$$S_k(n) = \sum_{i=1}^{k+2} S_k(i) \prod_{j \neq i} \frac{n-j}{i-j}$$

然后对于 $\prod n-j$ 预处理一下，发现 $\prod i-j$ 实际上一个数的阶乘乘上 (-1) 的一个数次方，分别算一下就可以 $O(k \log k)$ 了。

五. 组合数学

I. 组合数的基本性质

排列数与组合数

杨辉三角、组合数的一些恒等式、二项式定理

定义牛顿二项式系数

$$\binom{r}{n} = \begin{cases} 0 & (n < 0) \\ 1 & (n = 0) \\ \frac{r(r-1)\cdots(r-n+1)}{n!} & (n > 0) \end{cases}$$

注意其中的 $n \in \mathbb{Z}$ ，而 $r \in \mathbb{R}$ ，所以牛顿二项式系数没有组合意义，当 r, n 均为非负整数时， $\binom{r}{n}$ 在数值上等于 C_r^n 。

牛顿二项式定理

$$(x+y)^\alpha = \sum_{n=0}^{\infty} \binom{\alpha}{n} x^n y^{\alpha-n}$$

其中 $\alpha \in \mathbb{R}$ 。（一种可行的证明是使用泰勒级数将左式展开）

- 组合数的上指标翻转

$$\binom{r}{n} = (-1)^n \binom{n-r-1}{n}$$

范德蒙德卷积

$$\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}$$

卡特兰数

想象成在二维坐标网格上从 $(0, 0)$ 沿对角线游走到 $(n, 0)$ 的方案数，不能走到第三象限。

考虑容斥即可得到方案数为 $\binom{2n}{n} - \binom{2n}{n-1}$ 。

II. 容斥原理

利用容斥原理设计的 dp 举例：

- 带标号有向无环图计数

考虑设 f_n 代表 n 的点带标号有向无环图数量。

注意到有向无环图中一定会有至少一个入度为 0 的点。设 S_i 代表结点 i 入度为 0 的 DAG 集合，则 $f_n = |S_1 \cup S_2 \cup \dots \cup S_n|$ 。对 f_n 进行容斥，枚举钦定入度为 0 的点的个数 i ，考虑每个入度为 0 的点向剩余的 $n - i$ 个点是否连边，能推出如下等式

$$f_n = \sum_{i=1}^n (-1)^{i-1} \binom{n}{i} 2^{(n-i)i} f_{n-i}$$

直接暴力递推能够做到 $O(n^2)$ 的时间复杂度。

注意到，如果令 $g_i = (-1)^{i-1} \binom{n}{i} 2^{(n-i)i}$ ，则 $f_n = \sum_{i=1}^n f_i g_{n-i}$ ， $f_0 = 1$ ，是半在线卷积

的标准形式，使用分治 FFT 或多项式求逆计算 f 可分别做到 $O(n \log^2 n)$ 和 $O(n \log n)$ 的优秀时间复杂度。

III. 二项式反演

- 形式一

若

$$f(n) = \sum_{i=0}^n \binom{n}{i} g(i) \quad (2.1)$$

则有

$$g(n) = \sum_{i=0}^n \binom{n}{i} (-1)^{n-i} f(i) \quad (2.2)$$

证明：将等式 2.1 带入式 2.2 中，则 2.2 的右边可化为

$$\begin{aligned} & \sum_{i=0}^n \binom{n}{i} (-1)^{n-i} \sum_{j=0}^i \binom{i}{j} g(j) \\ &= \sum_{j=0}^n g(j) \sum_{i=j}^n \binom{n}{i} \binom{i}{j} (-1)^{n-i} \\ &= \sum_{j=0}^n g(j) \sum_{i=j}^n \binom{n}{j} \binom{n-j}{i-j} (-1)^{n-i} \\ &= \sum_{j=0}^n \binom{n}{j} g(j) \sum_{i=j}^n \binom{n-j}{i-j} (-1)^{n-i} \\ &= \sum_{j=0}^n \binom{n}{j} g(j) \sum_{k=0}^{n-j} \binom{n-j}{k} (-1)^{n-j-k} \\ &= \sum_{j=0}^n \binom{n}{j} g(j) [n-j=0] \\ &= g(n) \end{aligned}$$

所以式 2.2 左边等于右边，等式成立。

- 形式二

若

$$f(n) = \sum_{i=n}^m \binom{i}{n} g(i) \quad (2.3)$$

则有

$$g(n) = \sum_{i=n}^m \binom{i}{n} (-1)^{i-n} f(i) \quad (2.4)$$

证明：方法与形式一类似，将 2.3 带入 2.4，则 2.4 的右边可化为

$$\begin{aligned}
& \sum_{i=n}^m \binom{i}{n} (-1)^{i-n} \sum_{j=i}^m \binom{j}{i} g(j) \\
&= \sum_{j=n}^m g(j) \sum_{i=n}^j \binom{j}{i} \binom{i}{n} (-1)^{i-n} \\
&= \sum_{j=n}^m g(j) \sum_{i=n}^j \binom{j}{n} \binom{j-n}{i-n} (-1)^{i-n} \\
&= \sum_{j=n}^m \binom{j}{n} g(j) \sum_{i=n}^j \binom{j-n}{i-n} (-1)^{i-n} \\
&= \sum_{j=n}^m \binom{j}{n} g(j) \sum_{k=0}^{j-n} \binom{j-n}{k} (-1)^k \\
&= \sum_{j=n}^m \binom{j}{n} g(j) [j-n=0] \\
&= g(n)
\end{aligned}$$

所以式 2.4 左边等于右边，等式成立。

III. 生成函数初步

概率生成函数 PGF

对于离散的概率生成函数

$$f_X(x) = \sum_k P(X = k)x^k$$

$$E(X) = \sum_k k \cdot P(X = k) = f'_X(1)$$

$$E(X^2) = \sum_k k^2 P(X = k) = \sum_k [k(k-1) + k] P(X = k) = f''_X(1) + f'_X(1)$$

考虑怎么求 $E(x^n)$ ，由于

$$\begin{aligned} f_X(e^x) &= \sum_k P(X = k)e^{kx} \\ &= \sum_k P(X = k) \sum_n \frac{(kx)^n}{n!} \\ &= \sum_n \frac{x^n}{n!} \sum_k P(X = k)k^n \\ &= \sum_n \frac{E(X^n)}{n!} x^n \end{aligned}$$

这称为随机变量 X 的矩母函数，由此可见 $E(x^n) = n![x^n]f_X(e^x)$ ，考虑怎么求 $f_X(e^x)$ 的各项系数。

若 $f_X(x)$ 是无限项的，要求其封闭形式；若 $f_X(x)$ 是有限项的，考虑求

$$\begin{aligned} \sum_n E(X^n)x^n &= \sum_n \sum_k P(X = k)k^n x^n \\ &= \sum_n \sum_k f_k k^n x^n \\ &= \sum_k f_k \sum_n (kx)^n \\ &= \sum_k \frac{f_k}{1 - kx} \end{aligned}$$

使用分治 FFT 维护分子分母，最后多项式求逆即可求出，时间复杂度 $O(n \log^2 n)$ 。

下面我们来一道例题，一个歌唱王国的加强版。

- 字符集为 Σ ，打出第 i 个字符的概率为 p_i ，给出一个字符串 s ，求后缀第一次出现字符串 s 的期望打字次数 $E(s)$ 。

令 f_k 代表打了 k 个字符还没停的概率， g_k 代表打了 k 个字符恰好停的概率。

由于 $f_k = \sum_{j=k+1} g_j = f_{k+1} + g_{k+1}$ ，即

$$1 + xf(x) = f(x) + g(x) \quad (1)$$

考虑打了 p 个字符还没停，并且钦定在后面完整地打出一个字符串 t 的概率，用两种不同的形式表达，即

$$f_p \prod_{i=1}^{|t|} p_{t_i} = \sum_{k \in \text{border}(t)} g_{p+k} \prod_{i=k+1}^{|t|} p_{t_i}$$

写成生成函数的形式，即

$$f(x) \cdot x^{|t|} \prod_{i=1}^{|t|} p_{t_i} = g(x) \sum_{k \in \text{border}(t)} x^{|t|-k} \prod_{i=k+1}^{|t|} p_{t_i} \quad (2)$$

将 (1) 式带入 (2) 式中，化简可得

$$g(x) = \frac{\prod_{i=1}^{|t|} p_{t_i}}{(1-x) \sum_{k \in \text{border}(t)} x^{-k} \prod_{i=k+1}^{|t|} p_{t_i} + \prod_{i=1}^{|t|} p_{t_i}}$$

求导带入 $x = 1$ 即可得到

$$E(s) = g'(1) = \sum_{k \in \text{border}(t)} \prod_{i=1}^k \frac{1}{p_{t_i}}$$

IV. 十二重计数法

先放代码。

```
1 struct Twelve_count_method {
2     static mint Subtask1(int n, int m) { // 球之间互不相同，盒子之间互不相同，
    无放球限制
3         return (mint)m ^ n;
4     }
5     static mint Subtask2(int n, int m) { // 球之间互不相同，盒子之间互不相同，
    每个盒子至多装一个
6         return n > m ? 0 : C(m, n) * fac[n];
7     }
8     static mint Subtask3(int n, int m) { // 球之间互不相同，盒子之间互不相同，
    每个盒子至少装一个球
9         return Subtask6(n, m) * fac[m];
10    }
11    static mint Subtask4(int n, int m) { // 球之间互不相同，盒子全部相同，无放
    球限制
12        vector<mint> f(m + 1), g(m + 1);
13        for (int i = 0; i <= m; i++) {
14            f[i] = ((i & 1) ? (Mod - 1) : 1) * invf[i];
15            g[i] = (mint)i ^ n * invf[i];
16        }
17        auto h = convolution(f, g);
18
19        mint res = 0;
20        for (int i = 0; i <= m; i++) {
21            res += h[i];
22        }
23        return res;
24    }
25    static mint Subtask5(int n, int m) { // 球之间互不相同，盒子全部相同，每个
    盒子至多装一个球
26        return n > m ? 0 : 1;
27    }
28    static mint Subtask6(int n, int m) { // 球之间互不相同，盒子全部相同，每个
    盒子至少装一个球
29        vector<mint> f(m + 1), g(m + 1);
30        for (int i = 0; i <= m; i++) {
31            f[i] = ((i & 1) ? (Mod - 1) : 1) * invf[i];
32            g[i] = (mint)i ^ n * invf[i];
33        }
34        auto h = convolution(f, g);
35        return h[m];
36    }
```

```

37     static mint Subtask7(int n, int m) { // 球全部相同，盒子之间互不相同，无放
球限制
38         return C(n + m - 1, m - 1);
39     }
40     static mint Subtask8(int n, int m) { // 球全部相同，盒子之间互不相同，每个
盒子至多装一个球
41         return C(m, n);
42     }
43     static mint Subtask9(int n, int m) { // 球全部相同，盒子之间互不相同，每个
盒子至少装一个球
44         return C(n - 1, m - 1);
45     }
46     static mint Subtask10(int n, int m) { // 球全部相同，盒子全部相同，无放球限
制
47         vector<mint> f(n + 1, (mint)0);
48         for (int i = 1; i <= m; i++) {
49             for (int j = 1; i * j <= n; j++) {
50                 f[i * j] += inv[j];
51             }
52         }
53         f = polyexp(f, n + 1);
54         return f[n];
55     }
56     static mint Subtask11(int n, int m) { // 球全部相同，盒子全部相同，每个盒子
至多装一个球
57         return n > m ? 0 : 1;
58     }
59     static mint Subtask12(int n, int m) { // 球全部相同，盒子全部相同，每个盒子
至少装一个球
60         if (n < m) {
61             return 0;
62         }
63         return Subtask10(n - m, m);
64     }
65 };

```

第六重——第二类斯特林数

- 求方案数：将 n 个带标号的小球放入 m 个不带标号的盒子、要求盒子非空。

1. 递推式

设 $\left\{ \begin{smallmatrix} n \\ m \end{smallmatrix} \right\}$ 表示上述方案数，讨论 n 号小球是否单独放在一个新的盒子里，将 n 个小球的情况与 $n - 1$ 个小球的情况建立映射关系，能得到递推式

$$\left\{ \begin{smallmatrix} n \\ m \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n-1 \\ m-1 \end{smallmatrix} \right\} + m \left\{ \begin{smallmatrix} n-1 \\ m \end{smallmatrix} \right\} \quad (6.1)$$

2. 通项式

根据组合意义，能够得到

$$m^n = \sum_{i=0}^m \binom{m}{i} \left\{ \begin{matrix} n \\ i \end{matrix} \right\} i! \quad (6.2)$$

等式左边的含义：将 n 个带标号的小球，放到 m 个带标号的盒子中，盒子可以空的方案数；

等式右边的含义：先枚举非空盒子的个数 i ，然后计算 n 个带标号的小球放入 i 个带标号的盒子、要求盒子非空的方案数，与选出 i 个盒子的方案数相乘，求和后与等式左边的含义相同。

令 $f(i) = i^n, g(i) = \left\{ \begin{matrix} n \\ i \end{matrix} \right\} i!$ ，则等式 6.2 可写作

$$f(m) = \sum_{i=0}^m \binom{m}{i} g(i)$$

由二项式反演，得

$$g(m) = \sum_{i=0}^m \binom{m}{i} (-1)^{m-i} f(i)$$

展开后移项能够得到通项式

$$\left\{ \begin{matrix} n \\ m \end{matrix} \right\} = \frac{1}{m!} \sum_{i=0}^m \binom{m}{i} (-1)^{m-i} i^n \quad (6.3)$$

3. 行的计算

显然直接用递推式暴力计算需要 $O(n^2)$ 的时间复杂度，这样多耗费了很多时间代价去计算前 $n-1$ 行的第二类斯特林数，那有没有办法不计算前 $n-1$ 行的值就能得到第 n 行呢，请看下文。

将通项式中的组合数展开可以得到

$$\left\{ \begin{matrix} n \\ m \end{matrix} \right\} = \sum_{i=0}^m \frac{(-1)^{m-i}}{(m-i)!} \cdot \frac{i^n}{i!} \quad (6.4)$$

令 $F_i = \frac{(-1)^i}{i!}, G_i = \frac{i^n}{i!}$ ，容易发现，第 n 行的斯特林数数列 $\{S_{n,m}\}$ 就是 F 与 G 的卷积，可以利用快速数论变换进行优化，时间复杂度可以做到 $O(n \log n)$ 。

4. 列的计算

考虑指数生成函数 $F(x) = \sum_{n=1}^{\infty} \frac{x^n}{n!}$ ，通过生成函数的学习可以知道 $F^k(x)$ 是 i 个有标号小球放到 k 个有标号盒子中，且盒子不能为空的指数型生成函数，那么只需要将对应项的系数除以 $k!$ ，即可得到第二类斯特林数。即

$$\left\{ \begin{matrix} i \\ k \end{matrix} \right\} = \frac{i! [x^i] F^k(x)}{k!}$$

使用多项式快速幂计算出 $F^k(x)$ 即可在 $O(n \log n)$ 的时间复杂度内求出某一系列的前 n 个第二类斯特林数。

5. 应用

如果将等式 6.2 的组合数展开，写作下降幂的形式

$$m^n = \sum_{i=0}^m \left\{ \begin{matrix} n \\ i \end{matrix} \right\} m^{\underline{i}}$$

由于只有 n 个球，所以非空的盒子不可能超过 n 个，所以上式也可写作

$$m^n = \sum_{i=0}^{\min(n,m)} \left\{ \begin{matrix} n \\ i \end{matrix} \right\} m^{\underline{i}}$$

考虑求自然数的 k 次方和 $\sum_{i=1}^n i^k$ ，直接做是 $O(n \log k)$ 的，将 i^k 通过上式展开

$$\begin{aligned} \sum_{i=0}^n i^k &= \sum_{i=0}^n \sum_{j=0}^{\min(i,k)} \left\{ \begin{matrix} k \\ j \end{matrix} \right\} i^{\underline{j}} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \sum_{i=j}^n i^{\underline{j}} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} j! \sum_{i=j}^n \binom{i}{j} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} j! \binom{n+1}{j+1} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \frac{(n+1)^{\underline{j+1}}}{j+1} \end{aligned}$$

发现时间复杂度的瓶颈为求出第 k 行的第二类斯特林数，通过「3. 行的计算」中的方法，可以做到 $O(k \log k)$ 。（[题目链接](#)）

如果模数不是 NTT 模数，可以学习任意模数 NTT，也可以用拉格朗日插值来做。由于是多项式技巧，这里不展开写了，详见我的[这篇文章](#)。

六. 计算几何

I. 向量基础运算

```
1  const double eps = 1e-8;
2  const double pi = acos(-1);
3
4  bool dcmp(double x, double y = 0) {
5      return fabs(x - y) <= eps;
6  }
7
8  struct Pt {
9      double x, y;
10     Pt(double xx = 0, double yy = 0): x(xx), y(yy) {}
11     friend Pt operator + (const Pt& a, const Pt& b) {
12         Pt res(a.x + b.x, a.y + b.y);
13         return res;
14     }
15     friend Pt operator - (const Pt& a, const Pt& b) {
16         Pt res(a.x - b.x, a.y - b.y);
17         return res;
18     }
19     friend Pt operator * (const Pt& a, const double& k) {
20         Pt res(a.x * k, a.y * k);
21         return res;
22     }
23     friend Pt operator / (const Pt& a, const double& k) {
24         Pt res(a.x / k, a.y / k);
25         return res;
26     }
27 };
28
29 double dist(const Pt& a, const Pt& b) {
30     return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
31 }
32 double dot(const Pt& a, const Pt& b) {
33     return a.x * b.x + a.y * b.y;
34 }
35 double cross(const Pt& a, const Pt& b) {
36     return a.x * b.y - b.x * a.y;
37 }
```

II. 多边形

```
1 struct Poly {
2     vector<Pt> pts;
3     double area() {          // 顶点逆时针存储, 结果为正
4         double res = 0;
5         for (int i = 0; i < SZ(pts); i++) {
6             res += cross(pts[i], pts[(i + 1) % SZ(pts)]);
7         }
8         return res / 2;
9     }
10    double cir() {
11        double res = 0;
12        for (int i = 0; i < SZ(pts); i++) {
13            res += dist(pts[i], pts[(i + 1) % SZ(pts)]);
14        }
15        return res;
16    }
17 };
```

Ⅲ. 二维凸包

Andrew 算法求凸包模板。

```
1 Poly andrew(vector<Pt> p, int n) {
2     sort(p.begin(), p.end(), [&](auto u, auto v) -> bool {
3         if (dcmp(u.x, v.x)) {
4             return u.y > v.y;
5         }
6         return u.x < v.x;
7     });
8     int tp = 0;
9     vector st(n, 0);
10    vector used(n, false);
11    st[++tp] = 0;
12    for (int i = 1; i < n; i++) {          // 求下凸壳
13        while (tp >= 2 && cross(p[st[tp]] - p[st[tp - 1]], p[i] -
14p[st[tp]]) <= 0) {
15            used[st[tp--]] = false;
16        }
17        used[i] = true;
18        st[++tp] = i;
19    }
20    // 下凸壳最左和最右分别为 p[0] 和 p[n - 1]
21    int R = tp;
22    for (int i = n - 2; i >= 0; i--) {    // 求上凸壳
23        if (!used[i]) {
24            while (tp > R && cross(p[st[tp]] - p[st[tp - 1]], p[i] -
25p[st[tp]]) <= 0) {
26                used[st[tp--]] = false;
27            }
28            used[i] = true;
29            st[++tp] = i;
30        }
31    }
32    // st[1] 和 st[tp] 都为 p[0]
33    Poly res;
34    for (int i = 2; i <= tp; i++) {
35        res.pts.push_back(p[st[i]]);
36    }
37    return res;
38 }
```

七. 多项式部分

I. 多项式求逆

多项式求逆原理

设 $G(x)$ 为 $F(x)$ 模 x^n 意义下的逆, 则

$$F(x)G(x) \equiv 1 \pmod{x^n}$$

设 $G'(x)$ 为 $F(x)$ 模 $x^{n/2}$ 意义下的逆, 则

$$F(x)G'(x) \equiv 1 \pmod{x^{n/2}}$$

联立上述二式, 得

$$\begin{aligned} G(x) &\equiv G'(x) && \pmod{x^{n/2}} \\ (G(x) - G'(x))^2 &\equiv 0 && \pmod{x^n} \\ G(x)^2 - 2G(x)G'(x) + G'(x)^2 &\equiv 0 && \pmod{x^n} \\ F(x)G(x)^2 - 2F(x)G(x)G'(x) + F(x)G'(x)^2 &\equiv 0 && \pmod{x^n} \\ G(x) - 2G'(x) + F(x)G'(x)^2 &\equiv 0 && \pmod{x^n} \\ G(x) &\equiv G'(x)(2 - F(x)G'(x)) && \pmod{x^n} \end{aligned}$$

II. 多项式 $\ln$

多项式 $\ln$ 原理

设 $G(x) = \ln A(x)$ ，则两边同时求导得

$$\begin{aligned}\frac{d}{dx} G(x) &= \frac{d}{dA} \ln A(x) \frac{d}{dx} A(x) \\ \frac{d}{dx} G(x) &= \frac{\frac{d}{dx} A(x)}{A(x)}\end{aligned}$$

直接对 $A(x)$ 求导和求逆，把两个东西乘起来再进行一次不定积分，就可以得到 $G(x)$ 了。

III. 多项式 exp

多项式 exp 原理

设 $B(x) \equiv e^{A(x)} \pmod{x^n}$ ，两边取对数得

$$\begin{aligned}\ln B(x) &\equiv A(x) && \pmod{x^n} \\ \ln B(x) - A(x) &\equiv 0 && \pmod{x^n}\end{aligned}$$

设 $F(z)$ 是定义域为多项式的函数，且 $F(G(x)) = \ln G(x) - A(x)$ 。

设 $B_0(x)$ 满足 $\ln B_0(x) - A(x) \equiv 0 \pmod{x^{n/2}}$ ，即 $F(B_0(x)) \equiv 0 \pmod{x^{n/2}}$ 。

根据牛顿迭代，若

$$B(x) = B_0(x) - \frac{F(B_0(x))}{F'(B_0(x))}$$

则 $F(B(x)) \equiv 0 \pmod{x^n}$ 一定成立。

而我们化简一下上式就有

$$B(x) = B_0(x) - \frac{\ln B_0(x) - A(x)}{\frac{1}{B_0(x)}} = B_0(x)(1 - \ln B_0(x) + A(x))$$

这样我们就可以从 $\pmod{x^{n/2}}$ 意义下的 exp 求出 $\pmod{x^n}$ 意义下的 exp，时间复杂度是

$$T(n) = T\left(\frac{n}{2}\right) + O(n \log n) = O(n \log n)$$

IV. 多项式封装化模板

支持

- 多项式卷积 [convolution] $O(n \log n)$
- 多项式加法/减法 [polyadd/polysub] $O(n)$
- 多项式求导/积分 [derivation/integrate] $O(n)$
- 多项式求逆 [polyinv] $O(n \log n)$
- 多项式求对数 [polyln] $O(n \log n)$
- 多项式 $\exp$ [polyexp] $O(n \log n)$
- 多项式快速幂 [polyksm] $O(n \log n)$

```
1 namespace Polynomial {
2     const int Mod = 998244353;
3     const int G = 3;
4
5     template<int mod>
6     class Modint {
7     private:
8         unsigned int x;
9     public:
10         Modint() = default;
11         Modint(unsigned int x): x(x) {}
12         friend istream& operator >> (istream& in, Modint& a) {return in >>
a.x;}
13         friend ostream& operator << (ostream& out, Modint a) {return out
<< a.x;}
14         friend Modint operator + (Modint a, Modint b) {return (a.x + b.x)
% mod;}
15         friend Modint operator - (Modint a, Modint b) {return (a.x - b.x +
mod) % mod;}
16         friend Modint operator * (Modint a, Modint b) {return 1ULL * a.x *
b.x % mod;}
17         friend Modint operator / (Modint a, Modint b) {return a * ~b;}
18         friend Modint operator ^ (Modint a, int b) {Modint ans = 1; for(;
b; b >>= 1, a *= a) if(b & 1) ans *= a; return ans;}
19         friend Modint operator ~ (Modint a) {return a ^ (mod - 2);}
20         friend Modint operator - (Modint a) {return (mod - a.x) % mod;}
21         friend Modint& operator += (Modint& a, Modint b) {return a = a +
b;}
22         friend Modint& operator -= (Modint& a, Modint b) {return a = a -
b;}
23         friend Modint& operator *= (Modint& a, Modint b) {return a = a *
b;}
```

```

24         friend Modint& operator /= (Modint& a, Modint b) {return a = a /
b;}
25         friend Modint& operator ^= (Modint& a, int b) {return a = a ^ b;}
26         friend Modint& operator ++ (Modint& a) {return a += 1;}
27         friend Modint operator ++ (Modint& a, int) {Modint x = a; a += 1;
return x;}
28         friend Modint& operator -- (Modint& a) {return a -= 1;}
29         friend Modint operator -- (Modint& a, int) {Modint x = a; a -= 1;
return x;}
30         friend bool operator == (Modint a, Modint b) {return a.x == b.x;}
31         friend bool operator != (Modint a, Modint b) {return !(a == b);}
32     };
33     typedef Modint<Mod> mint;
34
35     void init_convo(vector<int>& rev, int& lim, int n, int m) {
36         int s = 0;
37         for (lim = 1; lim <= n + m; lim <= 1, s++);
38         rev.resize(lim);
39         for (int i = 0; i < lim; i++) {
40             rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (s - 1));
41         }
42     }
43
44     void NTT(vector<mint>& f, vector<int>& rev, int lim, int type) {
45         f.resize(lim, 0);
46         for (int i = 0; i < lim; i++) {
47             if (i < rev[i]) {
48                 swap(f[i], f[rev[i]]);
49             }
50         }
51         for (int i = 1; i < lim; i <= 1) {
52             mint mul = (mint)G ^ ((Mod - 1) / (i << 1));
53             if (type == -1) {
54                 mul = ~mul;
55             }
56             for (int j = 0; j < lim; j += (i << 1)) {
57                 mint w = 1;
58                 for (int k = 0; k < i; k++, w *= mul) {
59                     mint x = f[j + k], y = w * f[j + k + i];
60                     f[j + k] = x + y;
61                     f[j + k + i] = x - y;
62                 }
63             }
64         }
65     }
66
67     vector<mint> convolution(vector<mint> f, vector<mint> g, int k1, int
k2, mint (*calc)(mint x, mint y)) {

```



```

68     int n = SZ(f) - 1, m = SZ(g) - 1, lim;
69     vector<int> rev;
70     init_convo(rev, lim, k1 * n, k2 * m);
71
72     NTT(f, rev, lim, 1);
73     NTT(g, rev, lim, 1);
74     vector<mint> h(lim);
75     for (int i = 0; i < lim; i++) {
76         h[i] = calc(f[i], g[i]);
77     }
78     NTT(h, rev, lim, -1);
79     mint invlim = ~(mint)lim;
80     for (int i = 0; i < lim; i++) {
81         h[i] = h[i] * invlim;
82     }
83     h.resize(k1 * n + k2 * m + 1);
84     return h;
85 }
86 vector<mint> convolution(const vector<mint>& f, const vector<mint>& g)
{
87     return convolution(f, g, 1, 1, [] (mint x, mint y) -> mint {
88         return x * y;
89     });
90 }
91
92 vector<mint> derivation(vector<mint> f) {
93     int n = SZ(f);
94     for (int i = 1; i < n; i++) {
95         f[i - 1] = f[i] * i;
96     }
97     f.resize(n - 1);
98     return f;
99 }
100 vector<mint> integrate(vector<mint> f) {
101     int n = SZ(f);
102     f.resize(n + 1, 0);
103     for (int i = n - 1; i >= 0; i--) {
104         f[i + 1] = f[i] / (i + 1);
105     }
106     f[0] = 0;
107     return f;
108 }
109
110 vector<mint> polyadd(const vector<mint>& f, const vector<mint>& g) {
111     int n = SZ(f), m = SZ(g);
112     int mx = max(n, m), mn = min(n, m);
113     vector<mint> h(mx);
114     for (int i = 0; i < mn; i++) {

```

```

115         h[i] = f[i] + g[i];
116     }
117     for (int i = mn; i < mx; i++) {
118         h[i] = (n > m) ? f[i] : g[i];
119     }
120     return h;
121 }
122 vector<mint> polysub(const vector<mint>& f, const vector<mint>& g) {
123     int n = SZ(f), m = SZ(g);
124     int mx = max(n, m), mn = min(n, m);
125     vector<mint> h(mx);
126     for (int i = 0; i < mn; i++) {
127         h[i] = f[i] - g[i];
128     }
129     for (int i = mn; i < mx; i++) {
130         h[i] = (n > m) ? f[i] : -g[i];
131     }
132     return h;
133 }
134
135 vector<mint> polyinv(vector<mint> f, int n) {    // 1 / f(x) (mod x^n)
136     f.resize(n);
137     if (n == 1) {
138         f[0] = ~f[0];
139         return f;
140     }
141     auto g = polyinv(f, (n + 1) >> 1);
142     g = convolution(f, g, 1, 2, [](mint x, mint y) -> mint {
143         return (2 - x * y) * y;
144     });
145     g.resize(n, 0);
146     return g;
147 }
148
149 vector<mint> polyln(vector<mint> f, int n) {    // ln f(x) (mod x^n) ,
f[0] = 1
150     f = integrate(convolution(derivation(f), polyinv(f, n)));
151     f.resize(n, 0);
152     return f;
153 }
154
155 vector<mint> polyexp(vector<mint> f, int n) {    // exp f(x) (mod x^n)
, f[0] = 0 | Newton's Method
156     f.resize(n, 0);
157     if (n == 1) {
158         f[0] = 1;
159         return f;
160     }

```

```

161     auto g0 = polyexp(f, (n + 1) >> 1);
162     auto g1 = polysub(polyln(g0, n), f);
163     auto h = convolution(g0, g1, 1, 1, [] (mint x, mint y) -> mint {
164         return x * (1 - y);
165     });
166     h.resize(n, 0);
167     return h;
168 }
169
170 vector<mint> __polyksm(vector<mint> f, int k, int n) {    //
[f(x)]^{k} (mod x^n) , f[0] = 0 | exp(ln f(x))
171     f = polyln(f, n);
172     for (int i = 0; i < n; i++) {
173         f[i] *= k;
174     }
175     f = polyexp(f, n);
176     f.resize(n, 0);
177     return f;
178 }
179
180 vector<mint> polyksm(vector<mint> f, int k, int n) {    // [f(x)]^{k}
(mod x^n)
181     f.resize(n, 0);
182     int p = 0;
183     for (int i = 0; i < n; i++) {
184         if (f[i] != 0) {
185             p = i;
186             break;
187         }
188     }
189     mint coef = ~f[p];
190     vector<mint> g(n - p);
191     for (int i = 0; i < n - p; i++) {
192         g[i] = f[i + p] * coef;
193     }
194     g = __polyksm(g, k, n);
195
196     ll d = 1ll * p * k;
197     coef = (~coef) ^ k;
198     fill(f.begin(), f.end(), 0);
199     for (int i = n - 1; i >= d; i--) {
200         f[i] = g[i - d] * coef;
201     }
202     return f;
203 }
204 }
205 using Polynomial::convolution;
206 using Polynomial::Mod;

```

```
207 using Polynomial::mint;
```

V. 任意模数 NTT

```
1 namespace Polynomial {
2     const int Mod[] = {469762049, 998244353, 1004535809};
3     const int G = 3;
4
5     template<int mod>
6     class Modint {
7     private:
8         unsigned int x;
9     public:
10         Modint() = default;
11         Modint(unsigned int x): x(x) {}
12         unsigned int getx() { return x; }
13         friend istream& operator >> (istream& in, Modint& a) {return in >>
a.x;}
14         friend ostream& operator << (ostream& out, Modint a) {return out
<< a.x;}
15         friend Modint operator + (Modint a, Modint b) {return (a.x + b.x)
% mod;}
16         friend Modint operator - (Modint a, Modint b) {return (a.x - b.x +
mod) % mod;}
17         friend Modint operator * (Modint a, Modint b) {return 1ULL * a.x *
b.x % mod;}
18         friend Modint operator / (Modint a, Modint b) {return a * ~b;}
19         friend Modint operator ^ (Modint a, int b) {Modint ans = 1; for(
b; b >= 1, a *= a) if(b & 1) ans *= a; return ans;}
20         friend Modint operator ~ (Modint a) {return a ^ (mod - 2);}
21         friend Modint operator - (Modint a) {return (mod - a.x) % mod;}
22         friend Modint& operator += (Modint& a, Modint b) {return a = a +
b;}
23         friend Modint& operator -= (Modint& a, Modint b) {return a = a -
b;}
24         friend Modint& operator *= (Modint& a, Modint b) {return a = a *
b;}
25         friend Modint& operator /= (Modint& a, Modint b) {return a = a /
b;}
26         friend Modint& operator ^= (Modint& a, int b) {return a = a ^ b;}
27         friend Modint& operator ++ (Modint& a) {return a += 1;}
28         friend Modint operator ++ (Modint& a, int) {Modint x = a; a += 1;
return x;}
29         friend Modint& operator -- (Modint& a) {return a -= 1;}
30         friend Modint operator -- (Modint& a, int) {Modint x = a; a -= 1;
return x;}
31         friend bool operator == (Modint a, Modint b) {return a.x == b.x;}
32         friend bool operator != (Modint a, Modint b) {return !(a == b);}
33     };
```

```

34
35     typedef Modint<469762049> mintp0;
36     typedef Modint<998244353> mintp1;
37     typedef Modint<1004535809> mintp2;
38
39     void init_convo(vector<int>& rev, int& lim, int n, int m) {
40         int s = 0;
41         for (lim = 1; lim <= n + m; lim <= 1, s++);
42         rev.resize(lim);
43         for (int i = 0; i < lim; i++) {
44             rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (s - 1));
45         }
46     }
47
48     template <typename mint>
49     void NTT(vector<mint>& f, vector<int>& rev, int lim, int type, int
mod) {
50         f.resize(lim, 0);
51         for (int i = 0; i < lim; i++) {
52             if (i < rev[i]) {
53                 swap(f[i], f[rev[i]]);
54             }
55         }
56         for (int i = 1; i < lim; i <= 1) {
57             mint mul = (mint)G ^ ((mod - 1) / (i << 1));
58             if (type == -1) {
59                 mul = ~mul;
60             }
61             for (int j = 0; j < lim; j += (i << 1)) {
62                 mint w = 1;
63                 for (int k = 0; k < i; k++, w *= mul) {
64                     mint x = f[j + k], y = w * f[j + k + i];
65                     f[j + k] = x + y;
66                     f[j + k + i] = x - y;
67                 }
68             }
69         }
70     }
71
72     template <typename mint>
73     vector<mint> convolution(vector<mint> f, vector<mint> g, int mod) {
74         int n = SZ(f) - 1, m = SZ(g) - 1, lim;
75         vector<int> rev;
76         init_convo(rev, lim, n, m);
77
78         NTT(f, rev, lim, 1, mod);
79         NTT(g, rev, lim, 1, mod);
80         vector<mint> h(lim);

```

```

81     for (int i = 0; i < lim; i++) {
82         h[i] = f[i] * g[i];
83     }
84     NTT(h, rev, lim, -1, mod);
85     mint invlim = ~(mint)lim;
86     for (int i = 0; i < lim; i++) {
87         h[i] = h[i] * invlim;
88     }
89     h.resize(n + m + 1);
90     return h;
91 }
92
93 ll ksm(ll x, ll y, ll mod) {
94     ll res = 1;
95     while (y) {
96         if (y & 1) {
97             res = res * x % mod;
98         }
99         x = x * x % mod;
100        y >>= 1;
101    }
102    return res;
103 }
104 ll inv(ll x, ll mod) {
105     return ksm(x, mod - 2, mod);
106 }
107 ll mul(ll a, ll b, ll mod) {
108     return (a * b - (ll)((long double)a / mod * b) * mod + mod) % mod;
109 }
110
111 vector<int> convolution(const vector<int>& f, const vector<int>& g,
112 int mod) {
113     int n = SZ(f) - 1, m = SZ(g) - 1;
114     vector res(3, vector(n + m + 1, 0));
115
116     {
117         vector<mintp0> F, G;
118         F.resize(n + 1);
119         G.resize(m + 1);
120         for (int i = 0; i <= n; i++) {
121             F[i] = f[i];
122         }
123         for (int i = 0; i <= m; i++) {
124             G[i] = g[i];
125         }
126         auto cur = convolution(F, G, Mod[0]);
127         for (int i = 0; i <= n + m; i++) {
128             res[0][i] = cur[i].getx();
129         }
130     }
131 }

```

```

128         }
129     }
130
131     {
132         vector<mintp1> F, G;
133         F.resize(n + 1);
134         G.resize(m + 1);
135         for (int i = 0; i <= n; i++) {
136             F[i] = f[i];
137         }
138         for (int i = 0; i <= m; i++) {
139             G[i] = g[i];
140         }
141         auto cur = convolution(F, G, Mod[1]);
142         for (int i = 0; i <= n + m; i++) {
143             res[1][i] = cur[i].getx();
144         }
145     }
146
147     {
148         vector<mintp2> F, G;
149         F.resize(n + 1);
150         G.resize(m + 1);
151         for (int i = 0; i <= n; i++) {
152             F[i] = f[i];
153         }
154         for (int i = 0; i <= m; i++) {
155             G[i] = g[i];
156         }
157         auto cur = convolution(F, G, Mod[2]);
158         for (int i = 0; i <= n + m; i++) {
159             res[2][i] = cur[i].getx();
160         }
161     }
162
163     vector<int> h;
164     h.resize(n + m + 1);
165     ll p0 = Mod[0], p1 = Mod[1], p2 = Mod[2];
166     ll k0 = inv(p1, p0), k1 = inv(p0, p1);
167     ll ip0 = inv(p0, p2), ip1 = inv(p1, p2);
168     for (int i = 0; i <= n + m; i++) {
169         ll a0 = res[0][i], a1 = res[1][i], a2 = res[2][i];
170         ll m = p0 * p1;
171         ll a3 = (mul(a0, mul(k0, p1, m), m) + mul(a1, mul(k1, p0, m),
m)) % m;
172         ll a4 = mul(mul(((a2 - a3) % p2 + p2) % p2, ip0, p2), ip1,
p2);
173         h[i] = (mul(mul(a4, p0, mod), p1, mod) + a3) % mod;

```



```
174         }  
175         return h;  
176     }  
177 }  
178 using Polynomial::convolution;
```

MISC (杂项)

mt19937-随机数

```
1 mt19937 mrnd(std::chrono::steady_clock::now().time_since_epoch().count());
2 int rnd(int l, int r) {
3     return mrnd() % (r - l + 1) + l;
4 }
5 double rnd01() {
6     return mrnd() * 1.0 / UINT32_MAX;
7 }
```

扫描线

```
1 struct Line {
2     Line(int l, int r, int v): l(l), r(r), v(v) {}
3     int l, r, v;
4 };
5 struct Scan_Line {
6     Seg_T tr;
7     vector<Line> line[MAXN];
8     void add(int l0, int r0, int v) {
9         line[l0].push_back(Line(l0, r0, v));
10        line[r0 + 1].push_back(Line(l0, r0, inv(v)));
11    }
12    int solve(int n) {
13        int res = 0;
14        tr.build(1, 1, n);
15        for (int i = 1; i <= n; i++) {
16            for (auto [l, r, v] : line[i]) {
17                tr.upd(1, 1, n, l, r, v);
18            }
19            res = (res + tr.query(1, 1, n, i, n)) % Mod;
20        }
21        return res;
22    }
23 } scan;
```

Fwt 快速沃尔什变换

```
1 namespace Fwt {
2     template<typename T>
3     void or_fwt(vector<T>& a, int lim, int type) {
4         a.resize(lim, 0);
5         for (int i = 1; i < lim; i <= 1) {
6             for (int j = 0; j < lim; j += (i < 1)) {
7                 for (int k = 0; k < i; k++) {
8                     a[j + k + i] += a[j + k] * type;
9                 }
10            }
11        }
12    }
13    template<typename T>
14    void and_fwt(vector<T>& a, int lim, int type) {
15        a.resize(lim, 0);
16        for (int i = 1; i < lim; i <= 1) {
17            for (int j = 0; j < lim; j += (i < 1)) {
18                for (int k = 0; k < i; k++) {
19                    a[j + k] += a[j + k + i] * type;
20                }
21            }
22        }
23    }
24    template<typename T>
25    void xor_fwt(vector<T>& a, int lim, int type) {
26        a.resize(lim, 0);
27        for (int i = 1; i < lim; i <= 1) {
28            for (int j = 0; j < lim; j += (i < 1)) {
29                for (int k = 0; k < i; k++) {
30                    T x = a[j + k], y = a[j + k + i];
31                    a[j + k] = x + y;
32                    a[j + k + i] = x - y;
33                    if (type == -1) {
34                        a[j + k] /= 2;
35                        a[j + k + i] /= 2;
36                    }
37                }
38            }
39        }
40    }
41
42    template<typename T>
43    vector<T> convo(vector<T> f, vector<T> g, string opt) {
44        assert(opt == "or" || opt == "and" || opt == "xor");
```

```

45     int n = SZ(f), m = SZ(g);
46     int lim = 1 << (__lg(max(n, m)) + 1);
47     if (opt == "or") {
48         or_fwt(f, lim, 1);
49         or_fwt(g, lim, 1);
50     } else if (opt == "and") {
51         and_fwt(f, lim, 1);
52         and_fwt(g, lim, 1);
53     } else {
54         xor_fwt(f, lim, 1);
55         xor_fwt(g, lim, 1);
56     }
57     vector<T> h(lim);
58     for (int i = 0; i < lim; i++) {
59         h[i] = f[i] * g[i];
60     }
61     if (opt == "or") {
62         or_fwt(h, lim, -1);
63     } else if (opt == "and") {
64         and_fwt(h, lim, -1);
65     } else {
66         xor_fwt(h, lim, -1);
67     }
68     return h;
69 }
70 }

```